

GPT PARA DESENVOLVEDORES WEB

Guia profissional de prompts, UI/UX,
engenharia de software e agentes

COMANDOS + ROTEIROS + EXEMPLOS COMPLEXOS



GUIA GPT PARA DESENVOLVEDORES WEB

Prompts, UI/UX, engenharia de software, agentes e Oracle APEX

Nota essencial Os comandos iniciados por / neste livro sao macros de prompt: convencoes reutilizaveis que voce pode colar em chats, Custom GPTs, agentes, IDEs ou arquivos de instrucoes. Eles nao sao comandos nativos oficiais do ChatGPT por si so.

Este manual foi desenhado para uso pratico. Cada card explica o que o comando faz, quando aplicar, quais entradas fornecer, um modelo de prompt, um exemplo de uso e os erros mais comuns. Os roteiros finais demonstram combinacoes completas para tarefas reais, incluindo analise, implementacao, validacao e criterios de aceite.

Regra de ouro Quanto mais o prompt descreve objetivo, contexto, restricoes, fonte da verdade e criterio de aceite, menor a chance de uma resposta generica ou de uma alteracao tecnicamente perigosa.

Como ler os exemplos

- Texto entre < > indica um valor que deve ser substituido pelo contexto real.
- Texto entre [] indica uma opcao ou bloco que pode ser incluido quando fizer sentido.
- Combine macros na primeira linha e, abaixo, escreva o briefing operacional.
- Em tarefas de edicao, informe explicitamente o que deve ser preservado: IDs, rotas, regras de negocio, contratos de API, banco e comportamento existente.
- Para agentes com acesso ao ambiente, exija inspecao antes da alteracao, validacao depois da alteracao e plano de rollback.

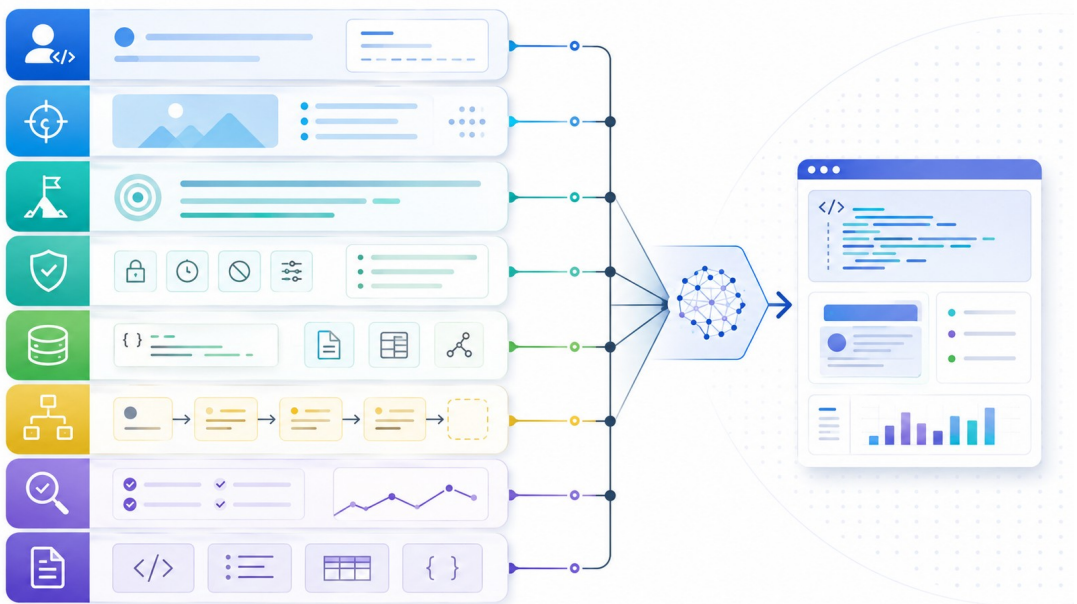
Mapa do livro

- | | |
|-----------|--|
| 01 | Fundamentos do prompt de engenharia |
| 02 | Comandos de contexto, escopo e controle |
| 03 | UI, UX e design de interfaces |
| 04 | HTML, CSS, JavaScript e componentizacao |
| 05 | Responsividade, acessibilidade e qualidade visual |
| 06 | APIs, backend, SQL e modelagem de dados |
| 07 | Seguranca, performance e observabilidade |
| 08 | Testes, debugging, code review e refatoracao |
| 09 | Arquitetura, Git, CI/CD e operacao |
| 10 | Oracle APEX - desenvolvimento assistido por GPT |
| 11 | Agentes de desenvolvimento e execucao segura |
| 12 | Roteiros completos e exemplos complexos |
| 13 | Biblioteca rapida de comandos e combinacoes |

CAPITULO 01

Fundamentos do prompt de engenharia

Um bom prompt de desenvolvimento funciona como uma especificação técnica compacta. Ele define papel, objetivo, contexto, limites, artefatos, fonte da verdade e testes de aceite. A inteligência deve receber uma tarefa verificável, não apenas um desejo vago.



Uma boa cadeia de prompt transforma contexto, restrições e critérios em uma saída verificável.

A anatomia de um prompt profissional

1. **Papel:** qual especialidade a IA deve assumir para raciocinar sobre a tarefa.
2. **Objetivo:** qual resultado de negócio ou técnico precisa ser alcançado.
3. **Contexto:** stack, arquitetura, página, arquivo, fluxo, usuário e problema atual.
4. **Fonte da verdade:** código, arquivos, banco, documentação, contratos e evidências que podem ser usados.
5. **Restrições:** o que não pode quebrar, alterar ou inventar.
6. **Plano:** ordem de análise, implementação, validação e rollback.
7. **Entregáveis:** código, diff, SQL, checklist, teste, documentação ou explicação.
8. **Crerios de aceite:** como comprovar que o trabalho terminou corretamente.

Modelo mental Trate o prompt como um ticket técnico + Definition of Done. A IA deve saber o estado inicial, o estado desejado e o método de validação.

/ATUAR-COMO

O QUE FAZ

Define a lente tecnica e o nivel de senioridade esperado.

QUANDO USAR

No inicio de tarefas que exigem criterio especializado ou decisoes de arquitetura.

ENTRADAS MINIMAS

Papel, stack, nivel de autonomia e dominio.

RESULTADO ESPERADO

Resposta orientada por criterios do dominio, com terminologia e trade-offs apropriados.

MODELO DE USO

```
/ATUAR-COMO: <papel tecnico>
Especialidade: <stack/domino>
Nivel: senior
Prioridade: precisao, seguranca e manutencao.
```

EXEMPLO COMPLETO

```
/ATUAR-COMO: Engenheiro Front-End senior e UX Engineer
Especialidade: HTML, CSS, JavaScript, Design Systems e acessibilidade
Prioridade: preservar comportamento existente e melhorar usabilidade.
```

Atencao Nao use o papel como substituto de requisitos. "Atue como especialista" sem contexto continua sendo um prompt fraco.

Combine com /OBJETIVO + /CONTEXTO + /CRITERIOS-ACEITE

/OBJETIVO

O QUE FAZ

Explicita o resultado final e evita que a IA otimize a coisa errada.

QUANDO USAR

Sempre que a tarefa tiver mais de uma interpretação possível.

ENTRADAS MINIMAS

Estado atual, estado desejado e impacto esperado.

RESULTADO ESPERADO

Uma meta mensurável que orienta todas as decisões seguintes.

MODELO DE USO

/OBJETIVO

Estado atual: <...>

Estado desejado: <...>

Sucesso significa: <...>

EXEMPLO COMPLETO

/OBJETIVO

Estado atual: formulário longo e confuso.

Estado desejado: cadastro dividido por grupos lógicos, responsivo e com ação principal evidente.

Sucesso significa: usuário concluir sem perder campos, regras ou validações existentes.

Atenção Evite objetivos puramente estéticos como "deixe bonito". Defina comportamento e resultado de uso.

Combine com /CRITERIOS-ACEITE + /UX-AUDIT

/CRITERIOS-ACEITE

O QUE FAZ

Converte expectativa em condicoes objetivas de conclusao.

QUANDO USAR

Em toda tarefa de implementacao, correcao, migracao ou refatoracao.

ENTRADAS MINIMAS

Lista de comportamentos que devem passar ao final.

RESULTADO ESPERADO

Checklist verificavel que limita mudancas e reduz retrabalho.

MODELO DE USO

/CRITERIOS-ACEITE

- <critério 1>
- <critério 2>
- <teste verificavel>

EXEMPLO COMPLETO

/CRITERIOS-ACEITE

- Nenhum ID existente pode mudar.
- Formulário deve funcionar em 360px, 768px e desktop.
- Validacoes atuais devem permanecer ativas.
- Não pode haver erro no console.
- Salvar e editar devem continuar persistindo os mesmos dados.

Atencao Critérios vagos como "ficar melhor" não são testáveis.

Combine com /TESTES + /VALIDAR-RESULTADO

CAPITULO 02

Comandos de contexto, escopo e controle

Estes comandos reduzem alucinacao, expansao indevida de escopo e alteracoes perigosas. Sao especialmente importantes quando a IA recebe codigo legado, acesso a um agente ou permissao para modificar um ambiente real.

/CONTEXTO

O QUE FAZ

Fornece o ambiente necessario para interpretar corretamente a tarefa.

QUANDO USAR

Quando stack, pagina, fluxo ou arquitetura influenciam a solucao.

ENTRADAS MINIMAS

Stack, versoes se relevantes, arquivos/paginas, comportamento atual.

RESULTADO ESPERADO

Analise aderente ao ambiente real.

MODELO DE USO

```
/CONTEXTO
Aplicacao: <...>
Stack: <...>
Pagina/arquivo: <...>
Comportamento atual: <...>
Dependencias: <...>
```

EXEMPLO COMPLETO

```
/CONTEXTO
Aplicacao administrativa web
Stack: Oracle APEX + ORDS
Pagina: formulario de cadastro
Comportamento atual: salva corretamente, mas layout e hierarquia estao ruins
Dependencias: Dynamic Actions e processos existentes.
```

Atencao Nao substitua a leitura do codigo/arquivo por uma descricao incompleta quando a fonte estiver disponivel.

Combine com /FONTE-DA-VERDADE + /PRESERVAR-FUNCIONALIDADE

/FONTE-DA-VERDADE

O QUE FAZ

Define quais evidencias podem sustentar decisoes e impede suposicoes.

QUANDO USAR

Ao trabalhar com arquivos, banco, API, documentacao ou sistema legado.

ENTRADAS MINIMAS

Arquivos, URLs internas, schema, logs ou trechos autorizados.

RESULTADO ESPERADO

Rastreabilidade entre conclusao e evidencia.

MODELO DE USO

/FONTE-DA-VERDADE

Use apenas: <fontes>.

Se algo nao estiver comprovado, marque como NAO CONFIRMADO e nao invente.

EXEMPLO COMPLETO

/FONTE-DA-VERDADE

Use HTML exportado da pagina, configuracao APEX fornecida e schema SQL anexado.

Nao invente Item Names, IDs, processos ou tabelas.

Atencao Nao trate memoria ou padroes comuns como prova da configuracao real.

Combine com /ANTI-ALUCINACAO

/RESTRICOES

O QUE FAZ

Delimita o que a solucao pode e nao pode fazer.

QUANDO USAR

Em sistemas produtivos, legado ou tarefas com risco de regressao.

ENTRADAS MINIMAS

Elementos imutaveis, tecnologias permitidas e limites de escopo.

RESULTADO ESPERADO

Mudanca de baixo risco e aderente ao escopo.

MODELO DE USO

```
/RESTRICOES
Nao alterar: <...>
Permitido: <...>
Proibido: <...>
```

EXEMPLO COMPLETO

```
/RESTRICOES
Nao alterar nomes de itens, IDs, processos, consultas ou regras de negocio.
Permitido ajustar CSS, classes e estrutura visual compativel.
Nao adicionar bibliotecas externas.
```

Atencao Restricao conflitante deve ser apontada, nao ignorada.

Combine com /PRESERVAR-FUNCIONALIDADE + /ROLLBACK

/ANTI-ALUCINACAO

O QUE FAZ

Obriga a IA a separar fato, inferencia e lacuna de informacao.

QUANDO USAR

Em tarefas com configuracao nao visivel, integracoes ou codigo incompleto.

ENTRADAS MINIMAS

Fontes disponiveis e politica para lacunas.

RESULTADO ESPERADO

Resposta mais confiavel e auditavel.

MODELO DE USO

/ANTI-ALUCINACAO

Nao invente <tipos>.

Classifique afirmacoes como CONFIRMADO, INFERIDO ou NECESSITA VERIFICACAO quando houver incerteza.

EXEMPLO COMPLETO

/ANTI-ALUCINACAO

Nao invente rotas, endpoints, nomes de tabela, IDs APEX ou parametros.

Se a pagina nao mostrar um valor, declare a ausencia e proponha como localizar.

Atencao Esse comando reduz, mas nao elimina, erros. Validacao tecnica continua obrigatoria.

Combine com /FONTE-DA-VERDADE + /VALIDAR-RESULTADO

/ANALISAR-ANTES

O QUE FAZ

Força uma etapa diagnostica antes de editar.

QUANDO USAR

Quando mudancas podem gerar regressao ou quando ha varias causas possiveis.

ENTRADAS MINIMAS

Artefato a analisar e pontos obrigatorios de inspecao.

RESULTADO ESPERADO

Plano fundamentado e menor chance de alteracao cosmetica quebrar logica.

MODELO DE USO

/ANALISAR-ANTES

1. Mapeie estrutura e dependencias.
2. Identifique causa/risco.
3. Defina menor alteracao segura.
4. So depois implemente.

EXEMPLO COMPLETO

/ANALISAR-ANTES

Antes de mover os botoes do formulario, identifique regioao, grid, template, classes, Dynamic Actions e condicoes de servidor que dependem deles.

Atencao Nao aceite uma analise que apenas repita o problema; ela deve apontar dependencias concretas.

Combine com /PLANO-EXECUCAO

/PLANO-EXECUCAO

O QUE FAZ

Organiza a tarefa em fases operacionais e pontos de controle.

QUANDO USAR

Projetos de varias etapas ou agentes com autonomia.

ENTRADAS MINIMAS

Objetivo, ordem, validacoes e rollback.

RESULTADO ESPERADO

Sequencia de trabalho controlada e verificavel.

MODELO DE USO

/PLANO-EXECUCAO

Fase 1: diagnostico

Fase 2: alteracao

Fase 3: testes

Fase 4: validacao

Fase 5: rollback se necessario

EXEMPLO COMPLETO

/PLANO-EXECUCAO

1. Inspecionar layout atual.
2. Registrar IDs/classes afetados.
3. Aplicar CSS isolado.
4. Testar desktop/mobile.
5. Testar salvar/cancelar.
6. Comparar antes/depois.

Atencao Plano sem criterios de parada permite que o agente continue alterando desnecessariamente.

Combine com /CRITERIOS-ACEITE + /ROLLBACK

/PRESERVAR-FUNCIONALIDADE

O QUE FAZ

Prioriza compatibilidade e regressao zero.

QUANDO USAR

Redesign, refatoracao e modernizacao de sistemas existentes.

ENTRADAS MINIMAS

Lista do que precisa continuar funcionando.

RESULTADO ESPERADO

Mudanca evolutiva em vez de reescrita arriscada.

MODELO DE USO

/PRESERVAR-FUNCIONALIDADE

Preserve integralmente: <contratos, IDs, fluxos, dados e regras>.

Nao altere backend sem necessidade comprovada.

EXEMPLO COMPLETO

/PRESERVAR-FUNCIONALIDADE

Mantenha Page Items, nomes, validacoes, branches, processos e Dynamic Actions. Foque inicialmente em layout, classes e propriedades visuais.

Atencao Preservar funcionalidade nao significa preservar todos os defeitos de UX; diferencie comportamento de apresentacao.

Combine com /TESTES + /ROLLBACK

CAPITULO 03

UI, UX e design de interfaces

A IA deve analisar uma interface como sistema de decisao: hierarquia, fluxo, feedback, estados, erros, consistencia e acessibilidade. "Deixar moderno" e insuficiente; o prompt precisa definir o que deve melhorar para o usuario.



Fluxo recomendado: descobrir, estruturar, desenhar, prototipar, validar acessibilidade, implementar e testar.

/WEBDESIGNER

O QUE FAZ

Ativa uma abordagem combinada de web design e front-end.

QUANDO USAR

Redesign, landing pages, paineis e sistemas administrativos.

ENTRADAS MINIMAS

Publico, objetivo da pagina, conteudo, stack e restricoes.

RESULTADO ESPERADO

Layout com hierarquia, consistencia e implementacao viavel.

MODELO DE USO

/WEBDESIGNER /UIUX-PRO /RESPONSIVO

Crie/reorganize <pagina> para <publico>. Preserve <restricoes>. Priorize <acoes>.

EXEMPLO COMPLETO

/WEBDESIGNER /UIUX-PRO /RESPONSIVO

Modernize a tela de cadastro administrativo. A acao primaria e Salvar; voltar e cancelar sao secundarias. Preserve todos os campos e IDs.

Atencao Nao aceite redesign que sacrifica densidade operacional ou esconde a acao principal.

Combine com /UX-AUDIT + /DESIGN-SYSTEM

/UX-AUDIT

O QUE FAZ

Faz diagnostico sistematico de usabilidade antes do redesign.

QUANDO USAR

Interfaces confusas, com retrabalho, muitos cliques ou erros de preenchimento.

ENTRADAS MINIMAS

Tela atual, tarefa do usuario, pontos de dor e fluxo.

RESULTADO ESPERADO

Lista priorizada de problemas + recomendacoes justificadas.

MODELO DE USO

/UX-AUDIT

Avalie: hierarquia, navegacao, carga cognitiva, feedback, erros, acessibilidade, responsividade e consistencia.

Classifique por severidade e impacto.

EXEMPLO COMPLETO

/UX-AUDIT

Analise este formulario de 30 campos. Identifique problemas P0/P1/P2, proponha agrupamentos, ordem de tabulacao e melhorias sem remover dados obrigatorios.

Atencao Evite auditoria baseada apenas em gosto visual. Vincule cada problema a uma consequencia de uso.

Combine com /FORM-UX + /CRITERIOS-ACEITE

/UI-MODERNA

O QUE FAZ

Define linguagem visual atual sem exageros decorativos.

QUANDO USAR

Quando a pagina parece datada ou inconsistente.

ENTRADAS MINIMAS

Design existente, identidade, densidade e tipo de usuario.

RESULTADO ESPERADO

Interface contemporanea com boa legibilidade.

MODELO DE USO

/UI-MODERNA

Aplique hierarquia, espacos, tipografia, estados, bordas e sombras discretas. Evite modismos que prejudiquem clareza.

EXEMPLO COMPLETO

/UI-MODERNA

Use superficies claras, radius moderado, divisores sutis, espaco 8pt e contrastes funcionais. Nada de glassmorphism em tabelas operacionais.

Atencao "Moderno" nao significa gradientes, sombras fortes e animacoes em todo lugar.

Combine com /DESIGN-SYSTEM

/DESIGN-SYSTEM

O QUE FAZ

Cria ou consolida tokens e componentes reutilizaveis.

QUANDO USAR

Projetos com varias telas, inconsistencias ou crescimento continuo.

ENTRADAS MINIMAS

Paleta, tipografia, espacamento, componentes e estados.

RESULTADO ESPERADO

Padroes previsiveis e menor custo de manutencao.

MODELO DE USO

/DESIGN-SYSTEM

Defina tokens de cor, espacamento, tipografia, radius, elevacao e estados. Depois mapeie componentes base.

EXEMPLO COMPLETO

/DESIGN-SYSTEM

Crie tokens --space-1..6, --radius-sm/md/lg, cores semanticas success/warning/danger e componentes button/input/card/table com estados.

Atencao Nao transforme tokens em nomes de cor fisica quando o significado semantico for mais estavel.

Combine com /COMPONENTIZAR + /ACESSIBILIDADE

/HIERARQUIA-VISUAL

O QUE FAZ

Ordena informacao pela importancia para a tarefa.

QUANDO USAR

Paginas onde tudo parece ter o mesmo peso.

ENTRADAS MINIMAS

Objetivo principal, informacoes criticas e acoes.

RESULTADO ESPERADO

Leitura mais rapida e orientacao imediata.

MODELO DE USO

/HIERARQUIA-VISUAL

Defina nivel 1: <...>; nivel 2: <...>; nivel 3: <...>. Ajuste tamanho, contraste e agrupamento sem ocultar informacao.

EXEMPLO COMPLETO

/HIERARQUIA-VISUAL

Nivel 1: numero do caso e status.

Nivel 2: envolvidos e risco.

Nivel 3: metadados e historico.

Acao primaria: Salvar.

Atencao Hierarquia nao deve depender apenas de cor; use tamanho, posicao, espaco e texto.

Combine com /CARDS + /BUTTON-UX

/CARDS

O QUE FAZ

Agrupa informacoes relacionadas em unidades escaneaveis.

QUANDO USAR

Dashboards e formularios com grupos naturais.

ENTRADAS MINIMAS

Grupos de conteudo, prioridades e acoes por grupo.

RESULTADO ESPERADO

Estrutura visual modular sem fragmentacao excessiva.

MODELO DE USO

/CARDS

Converta somente agrupamentos com identidade funcional em cards. Cada card deve ter titulo, objetivo e conteudo coerente.

EXEMPLO COMPLETO

/CARDS

Card 1: Dados da vitima; Card 2: Dados do agressor; Card 3: Fatores de risco. Nao crie card separado para cada campo.

Atencao Card para tudo vira ruido e aumenta a altura da pagina.

Combine com /HIERARQUIA-VISUAL

/FORM-UX

O QUE FAZ

Otimiza preenchimento, validacao e recuperacao de erro.

QUANDO USAR

Cadastros, pesquisas, filtros, wizard e edicao.

ENTRADAS MINIMAS

Campos, dependencias, obrigatoriedade, validacoes e frequencia de uso.

RESULTADO ESPERADO

Formulario mais rapido e menos propenso a erro.

MODELO DE USO

/FORM-UX

Agrupe campos por tarefa, ordene pelo fluxo mental, use labels persistentes, explique erros e preserve dados apos falha.

EXEMPLO COMPLETO

/FORM-UX

Nome/CPF/data no primeiro grupo; endereco no segundo; dados complementares recolhidos. CPF valida no blur e explica formato sem apagar o valor digitado.

Atencao Placeholder nao substitui label. Nao esconda campo obrigatorio sem deixar clara a condicao que o ativa.

Combine com /VALIDACAO-FRONTEND + /ACESSIBILIDADE

/TABLE-UX	
O QUE FAZ Melhora leitura e operacao em tabelas densas.	QUANDO USAR Listagens, relatórios, grids e telas administrativas.
ENTRADAS MINIMAS Colunas, prioridades, filtros, ordenacao e acoes.	RESULTADO ESPERADO Tabela escaneavel e operacional.
MODELO DE USO <code>/TABLE-UX</code> Priorize colunas, defina alinhamento, filtros, acoes por linha e comportamento responsivo.	
EXEMPLO COMPLETO <code>/TABLE-UX</code> Fixe identificador e nome; status vira badge; datas alinhadas; acoes em menu contextual; em mobile, colunas secundarias vao para detalhe expansivel.	
Atencao Nao use rolagem horizontal como unica estrategia de responsividade sem avaliar prioridade de colunas.	
Combine com <code>/RESPONSIVO + /ACESSIBILIDADE</code>	

/BUTTON-UX

O QUE FAZ

Define hierarquia e estados das acoes.

QUANDO USAR

Formularios, modais e telas com muitas acoes.

ENTRADAS MINIMAS

Acao primaria, secundarias, destrutivas e estados.

RESULTADO ESPERADO

Acoes compreensiveis e menos erros.

MODELO DE USO

/BUTTON-UX

Classifique cada acao como primaria, secundaria, neutra ou destrutiva. Inclua hover, focus, disabled e loading.

EXEMPLO COMPLETO

/BUTTON-UX

Salvar = primario. Cancelar = neutro. Excluir = danger com confirmacao. Durante salvar, desabilite duplo envio e mostre loading.

Atencao Dois botoes com o mesmo destaque competem por atencao.

Combine com /FEEDBACK-UX

/FEEDBACK-UX

O QUE FAZ

Garante retorno claro para cada ação ou estado.

QUANDO USAR

Operações assíncronas, salvamento, erro e listas vazias.

ENTRADAS MINIMAS

Ações, tempos, erros e estados vazios.

RESULTADO ESPERADO

Usuário sabe o que aconteceu e o que fazer a seguir.

MODELO DE USO

/FEEDBACK-UX

Mapeie estado inicial, loading, sucesso, vazio, erro, parcial e indisponível.

EXEMPLO COMPLETO

/FEEDBACK-UX

Ao salvar: spinner no botão; sucesso: toast + estado persistido; erro: mensagem perto do campo ou resumo com link para o problema.

Atenção Toast sozinho não resolve erro de campo que exige correção local.

Combine com /MICROINTERAÇÕES + /ACESSIBILIDADE

/MICROINTERACOES

O QUE FAZ

Adiciona transicoes funcionais e feedback de baixa friccao.

QUANDO USAR

Hover, expansao, loading, drag, sucesso e mudancas de estado.

ENTRADAS MINIMAS

Componente, acao e duracao desejada.

RESULTADO ESPERADO

Interface responsiva e previsivel.

MODELO DE USO

/MICROINTERACOES

Use animacao apenas para explicar transicao, feedback ou continuidade. Respeite prefers-reduced-motion.

EXEMPLO COMPLETO

/MICROINTERACOES

Accordion 160-220ms; focus ring imediato; skeleton apenas em conteudo realmente assincrono; sem animacao decorativa em tabela.

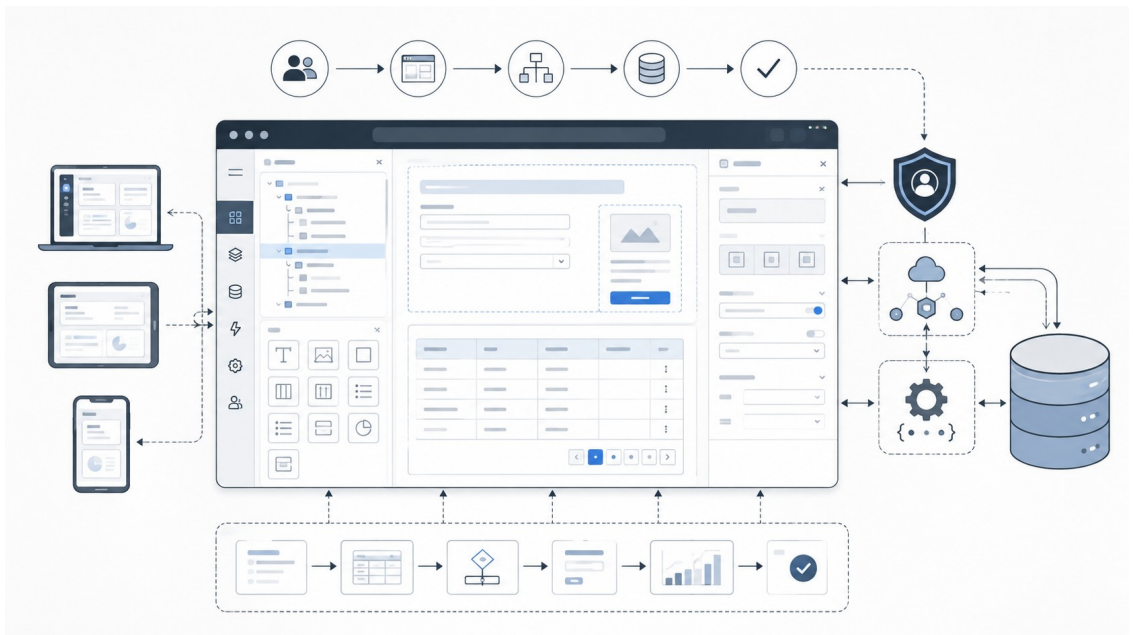
Atencao Animacao longa ou excessiva atrasa operacao.

Combine com /ACESSIBILIDADE

CAPITULO 04

HTML, CSS, JavaScript e componentizacao

O prompt de implementacao deve pedir estrutura semantica, isolamento de estilos, estados previsiveis e compatibilidade com o projeto existente. O objetivo nao e gerar o maior codigo possivel, e sim a menor solucao correta e sustentavel.



Uma interface madura conecta componentes, dados, segurança, integrações e comportamento responsivo sem acoplamento desnecessário.

/HTML-SEMANTICO

O QUE FAZ

Prioriza estrutura correta e acessivel.

QUANDO USAR

Criacao ou revisao de markup.

ENTRADAS MINIMAS

Conteudo, landmarks, formularios e interacoes.

RESULTADO ESPERADO

DOM mais robusto para acessibilidade, SEO e manutencao.

MODELO DE USO

/HTML-SEMANTICO

Use elementos nativos por significado. Evite div como botao, heading fora de ordem e ARIA redundante.

EXEMPLO COMPLETO

/HTML-SEMANTICO

Transforme o cabecalho em <header>, navegacao em <nav>, conteudo em <main>, acoes em <button> e campos com <label for>.

Atencao ARIA nao corrige HTML semanticamente errado quando existe elemento nativo apropriado.

Combine com /ACESSIBILIDADE

/CSS-ARQUITETURA

O QUE FAZ

Organiza estilos por tokens, componentes e escopo.

QUANDO USAR

CSS crescente, conflito de seletores ou legado.

ENTRADAS MINIMAS

Convencao atual, escopo e compatibilidade.

RESULTADO ESPERADO

CSS previsivel e menor risco de regressao.

MODELO DE USO

/CSS-ARQUITETURA

Use tokens e classes de componente. Evite !important salvo justificativa. Minimize especificidade e vazamento global.

EXEMPLO COMPLETO

/CSS-ARQUITETURA

Crie .case-form, .case-form__section, .case-form__actions e tokens locais; nao estilize input global se a aplicacao tem outras telas.

Atencao Aumentar especificidade em cascata para "ganhar" do tema cria divida tecnica.

Combine com /DESIGN-SYSTEM + /PRESERVAR-FUNCIONALIDADE

/JAVASCRIPT

O QUE FAZ

Pede logica de cliente clara e com tratamento de estados.

QUANDO USAR

Interacoes, validacao, chamadas assincronas e comportamento dinamico.

ENTRADAS MINIMAS

Eventos, dados, dependencias e navegadores alvo.

RESULTADO ESPERADO

Codigo cliente deterministico e resiliente.

MODELO DE USO

/JAVASCRIPT

Implemente <comportamento> com funcoes pequenas, tratamento de erro, cleanup e sem dependencias desnecessarias.

EXEMPLO COMPLETO

/JAVASCRIPT

Ao mudar UF, carregue municipios via fetch com AbortController; cancele requisicao anterior, mostre loading, trate erro e preserve selecao valida.

Atencao Nao duplique no cliente uma regra de segurança que precisa existir no servidor.

Combine com /VALIDACAO-FRONTEND + /TESTES

/TYPESCRIPT

O QUE FAZ

Modela contratos e reduz ambiguidades em código JS.

QUANDO USAR

Apps com estado, APIs e componentes reutilizáveis.

ENTRADAS MINIMAS

Tipos de domínio, payloads e regras de nullability.

RESULTADO ESPERADO

Erros detectados antes da execução e contratos mais claros.

MODELO DE USO

/TYPESCRIPT

Modele tipos de domínio e respostas de API. Evite any; diferencie estados válidos com unions.

EXEMPLO COMPLETO

/TYPESCRIPT

```
type LoadState<T> = {status:"idle"}|{status:"loading"}|{status:"success";data:T}|{status:"error";message:string};
```

Atencao Type assertion excessiva pode esconder o problema em vez de modelar a realidade.

Combine com /API-DESIGN + /TESTES

/COMPONENTIZAR

O QUE FAZ

Extraí unidades reutilizaveis sem fragmentar demais.

QUANDO USAR

Padroes repetidos ou componentes com estados proprios.

ENTRADAS MINIMAS

Responsabilidades, props/entradas e variacoes.

RESULTADO ESPERADO

Reuso com fronteiras claras.

MODELO DE USO

/COMPONENTIZAR

Identifique repeticao e responsabilidade. Extraia componentes coesos; evite componentes genericos prematuros.

EXEMPLO COMPLETO

/COMPONENTIZAR

Crie FieldGroup para agrupamento visual e StatusBadge para estados semanticos; nao crie componente para uma unica div sem comportamento.

Atencao Abstracao precoce aumenta complexidade.

Combine com /DESIGN-SYSTEM + /CODE-REVIEW

/STATE-MANAGEMENT

O QUE FAZ

Define fonte de verdade e transicoes de estado.

QUANDO USAR

Interfaces com dados sincronizados, filtros ou varios componentes.

ENTRADAS MINIMAS

Estados locais, remotos, derivados e persistidos.

RESULTADO ESPERADO

Menos inconsistencias e bugs de sincronizacao.

MODELO DE USO

/STATE-MANAGEMENT

Classifique estado em local, servidor, URL e derivado. Nao duplique estado calculavel.

EXEMPLO COMPLETO

/STATE-MANAGEMENT

Filtros compartilhaveis ficam na URL; dados remotos no cache de servidor; campo aberto/fechado local; total calculado deriva dos itens.

Atencao Guardar o mesmo dado em tres lugares cria divergencia.

Combine com /ARQUITETURA

/VALIDACAO-FRONTEND

O QUE FAZ

Melhora feedback sem substituir validacao de servidor.

QUANDO USAR

Formularios e fluxos com entrada do usuario.

ENTRADAS MINIMAS

Regras, mensagens e timing de validacao.

RESULTADO ESPERADO

Feedback rapido e seguranca preservada.

MODELO DE USO

/VALIDACAO-FRONTEND

Valide formato no cliente para UX e repita regras de integridade/autorizacao no servidor. Mensagens devem explicar como corrigir.

EXEMPLO COMPLETO

/VALIDACAO-FRONTEND

CPF: mascara e formato no cliente; existencia/permissoes no servidor; erro associado ao campo via aria-describedby.

Atencao Nunca confie apenas na validacao do navegador para regras de negocio.

Combine com /FORM-UX + /SECURITY-REVIEW

/PERFORMANCE-WEB

O QUE FAZ

Analisa custo de rede, renderizacao e runtime.

QUANDO USAR

Pagina lenta, bundle grande ou interacoes travadas.

ENTRADAS MINIMAS

Metricas, waterfall, bundle, DOM e fluxo.

RESULTADO ESPERADO

Plano de otimizacao baseado em evidencia.

MODELO DE USO

/PERFORMANCE-WEB

Meça antes. Classifique gargalo em rede, servidor, JS, renderizacao, imagens ou cache. Proponha mudancas por impacto.

EXEMPLO COMPLETO

/PERFORMANCE-WEB

Adie modulo de grafico fora da dobra, comprima imagens, reduza consultas duplicadas e evite re-render de tabela inteira a cada tecla.

Atencao Nao aplique memoizacao, lazy loading ou cache sem identificar o gargalo.

Combine com /OBSERVABILIDADE + /TESTES

/SEO-TECNICO

O QUE FAZ

Cobre estrutura indexavel e metadados tecnicos.

QUANDO USAR

Sites publicos, conteudo e landing pages.

ENTRADAS MINIMAS

Rotas, conteudo, canonical, robots e dados estruturados.

RESULTADO ESPERADO

Base tecnica para rastreamento e indexacao.

MODELO DE USO

/SEO-TECNICO

Revise titles, headings, canonical, robots, sitemap, links, dados estruturados e performance.

EXEMPLO COMPLETO

/SEO-TECNICO

Cada rota publica recebe title unico, H1 coerente, canonical, meta description e dados estruturados apenas quando semanticamente validos.

Atencao SEO nao e relevante para areas autenticadas internas e nao deve comprometer privacidade.

Combine com /PERFORMANCE-WEB

CAPITULO 05

Responsividade, acessibilidade e qualidade visual

Responsividade real nao e apenas "caber no celular". Ela precisa preservar prioridade, toque, leitura, foco, ordem de navegacao e completude da tarefa em diferentes larguras e modos de entrada.

/RESPONSIVO

O QUE FAZ

Adapta layout e conteudo a diferentes larguras.

QUANDO USAR

Qualquer interface usada em mais de um tamanho de tela.

ENTRADAS MINIMAS

Breakpoints por conteudo, prioridades e componentes.

RESULTADO ESPERADO

Tarefa completa em todos os tamanhos alvo.

MODELO DE USO

/RESPONSIVO

Projete mobile, tablet e desktop. Breakpoints devem responder ao conteudo, nao a modelos de aparelho.

EXEMPLO COMPLETO

/RESPONSIVO

Em 360px, filtros recolhem em painel e tabela mostra 3 colunas essenciais; em desktop, filtros laterais e 8 colunas.

Atencao Nao resolva tudo com scale ou fonte menor.

Combine com /MOBILE-FIRST + /TABLE-UX

/MOBILE-FIRST

O QUE FAZ

Prioriza conteudo e interacao antes de expandir layout.

QUANDO USAR

Produtos com uso relevante em smartphone ou restricao de espaco.

ENTRADAS MINIMAS

Acoes essenciais, toque e navegacao.

RESULTADO ESPERADO

Experiencia coerente e priorizada.

MODELO DE USO

/MOBILE-FIRST

Comece pelo fluxo essencial em 320-360px; expanda sem mudar significado.

EXEMPLO COMPLETO

/MOBILE-FIRST

Acao primaria ocupa largura util; campos empilham; elementos secundarios vao para accordion; desktop reorganiza em grid.

Atencao Mobile-first nao significa esconder dados importantes no mobile.

Combine com /RESPONSIVO

/ACESSIBILIDADE

O QUE FAZ

Incorpora operacao por teclado, contraste, semantica e comunicacao de estados.

QUANDO USAR

Todo produto; especialmente governo, servicos essenciais e sistemas internos amplos.

ENTRADAS MINIMAS

Fluxos, componentes, mensagens e midia.

RESULTADO ESPERADO

Interface utilizavel por mais pessoas e mais robusta tecnicamente.

MODELO DE USO

/ACESSIBILIDADE

Revise semantica, foco, teclado, contraste, nomes acessiveis, erros, reflow, zoom e reduced motion.

EXEMPLO COMPLETO

/ACESSIBILIDADE

Modal prende foco enquanto aberto, fecha com Esc, retorna foco ao gatilho; erro de campo e anunciado e visual; botoes possuem nome acessivel.

Atencao Nao dependa de cor, hover ou mouse para transmitir informacao.

Combine com /HTML-SEMANTICO + /TESTES

/DARK-MODE

O QUE FAZ

Adapta tokens para tema escuro mantendo contraste e hierarquia.

QUANDO USAR

Produtos que oferecem escolha de tema ou uso prolongado.

ENTRADAS MINIMAS

Tokens, superfícies, estados e imagens.

RESULTADO ESPERADO

Tema consistente sem duplicar CSS por componente.

MODELO DE USO

/DARK-MODE

Mapeie cores semanticas para light/dark; preserve contraste e evite preto puro em grandes superficies.

EXEMPLO COMPLETO

/DARK-MODE

background/surface/text/border/primary sao tokens; componentes nao codificam #fff diretamente.

Atencao Inverter cores automaticamente pode destruir significado e contraste.

Combine com /DESIGN-SYSTEM

/VISUAL-QA

O QUE FAZ

Pede inspecao sistematica de defeitos visuais.

QUANDO USAR

Antes de liberar redesign ou alteracoes CSS.

ENTRADAS MINIMAS

Telas, viewports e estados.

RESULTADO ESPERADO

Checklist visual de regressao.

MODELO DE USO

/VISUAL-QA

Verifique clipping, overflow, alinhamento, quebra, foco, estados vazios, loading, erros e zoom.

EXEMPLO COMPLETO

/VISUAL-QA

Testar 360x800, 768x1024 e 1440x900; zoom 200%; nomes longos; tabela vazia; erro de API; modal e teclado.

Atencao Happy path sozinho nao cobre layout real.

Combine com /E2E + /CRITERIOS-ACEITE

CAPITULO 06

APIs, backend, SQL e modelagem de dados

Nesta camada, o prompt deve explicitar contratos, validacao, idempotencia, transacoes, erros e propriedade do dado. A IA precisa saber o que e interface publica, o que e implementacao interna e quais garantias nao podem mudar.

/API-DESIGN

O QUE FAZ

Desenha contrato de API coerente e evolutivo.

QUANDO USAR

Novos endpoints ou revisao de API.

ENTRADAS MINIMAS

Recursos, operacoes, autorizacao, erros e consumidores.

RESULTADO ESPERADO

Contrato claro para clientes e testes.

MODELO DE USO

/API-DESIGN

Modele recursos, metodos, payloads, codigos de status, erros, paginacao, idempotencia e versionamento quando necessario.

EXEMPLO COMPLETO

/API-DESIGN

POST /cases cria; GET /cases/{id} le; PATCH altera parcialmente. Erros usam {code,message,details,traceId}. POST aceita Idempotency-Key.

Atencao Nao escolha status HTTP ou semantica apenas por conveniencia do backend.

Combine com /SECURITY-REVIEW + /TESTES

/INTEGRACAO-API

O QUE FAZ

Planeja consumo seguro e resiliente de servicos externos.

QUANDO USAR

Integrações REST/GraphQL/servicos corporativos.

ENTRADAS MINIMAS

Contrato, auth, timeout, retry e falhas.

RESULTADO ESPERADO

Integracao previsivel e diagnosticavel.

MODELO DE USO

/INTEGRACAO-API

Mapeie autenticacao, timeout, retry, rate limit, cache, erros e observabilidade. Nao exponha segredos ao frontend.

EXEMPLO COMPLETO

/INTEGRACAO-API

Servico de consulta: timeout 3s, retry apenas em falha transitória e operacao idempotente, circuit breaker no backend e traceId em log.

Atencao Retry indiscriminado pode duplicar operacoes ou agravar incidente.

Combine com /RESILIENCIA + /OBSERVABILIDADE

/BACKEND

O QUE FAZ

Estrutura regra de negocio e camada de aplicacao.

QUANDO USAR

Implementacao de endpoints, jobs e servicos.

ENTRADAS MINIMAS

Entidades, casos de uso, dependencias e persistencia.

RESULTADO ESPERADO

Codigo com responsabilidades claras.

MODELO DE USO

/BACKEND

Separe validacao de entrada, autorizacao, regra de negocio, persistencia e serializacao. Mantenha transacao onde necessario.

EXEMPLO COMPLETO

/BACKEND

Use caso ApproveCase verifica permissao, estado atual e invariantes; repositorio persiste dentro da transacao; controller apenas adapta HTTP.

Atencao Controller gordo e regra espalhada dificultam testes.

Combine com /ARQUITETURA + /UNIT-TEST

/MODELAGEM-DADOS

O QUE FAZ

Projeta entidades, relacoes e invariantes.

QUANDO USAR

Novo dominio, schema confuso ou migracao.

ENTRADAS MINIMAS

Regras de negocio, cardinalidade, historico e consulta.

RESULTADO ESPERADO

Schema que representa o dominio e suporta consultas reais.

MODELO DE USO

/MODELAGEM-DADOS

Modele entidades, chaves, cardinalidade, nulabilidade, unicidade, historico e padrao de acesso.

EXEMPLO COMPLETO

/MODELAGEM-DADOS

Case 1:N Participant; Participant tem role controlado; historico de status em CaseStatusHistory; CPF pode ser nulo mas, quando presente, deve obedecer unicidade de negocio definida.

Atencao Normalizar ou desnormalizar sem considerar padrao de acesso e integridade e uma decisao incompleta.

Combine com /SQL + /MIGRACAO

/SQL

O QUE FAZ

Gera ou revisa consultas com foco em corretude e custo.

QUANDO USAR

Relatorios, CRUD, performance e integridade.

ENTRADAS MINIMAS

Schema real, cardinalidade, filtros e indice existentes.

RESULTADO ESPERADO

Consulta correta, legivel e analisavel.

MODELO DE USO

/SQL

Use apenas tabelas/colunas confirmadas. Explique joins, filtros e risco de multiplicacao de linhas. Considere plano de execucao.

EXEMPLO COMPLETO

/SQL

Liste casos com ultimo status usando window function ou join controlado; evite subconsulta correlacionada N vezes se houver alternativa melhor para o SGBD.

Atencao Nao invente coluna nem indice. Sem schema, forneça pseudoconsulta marcada como tal.

Combine com /FONTE-DA-VERDADE + /PERFORMANCE-WEB

/MIGRACAO

O QUE FAZ

Planeja evolucao de schema com compatibilidade.

QUANDO USAR

Alteracoes de banco em sistemas ativos.

ENTRADAS MINIMAS

Estado atual, estado novo, volume e janela.

RESULTADO ESPERADO

Mudanca gradual e reversivel.

MODELO DE USO

/MIGRACAO

Produza expand-migrate-contract, backfill, validacao, rollback e estrategia para zero/baixo downtime.

EXEMPLO COMPLETO

/MIGRACAO

1 adicionar nova coluna nullable; 2 deploy que escreve antiga+nova; 3 backfill; 4 validar; 5 leitura nova; 6 tornar obrigatoria; 7 remover antiga em release posterior.

Atencao ALTER destrutivo imediato pode quebrar versoes antigas da aplicacao.

Combine com /ROLLBACK + /CI-CD

/CACHE

O QUE FAZ

Define o que pode ser armazenado e por quanto tempo.

QUANDO USAR

Leituras repetidas, APIs caras ou latencia alta.

ENTRADAS MINIMAS

Consistencia, TTL, chave e invalidacao.

RESULTADO ESPERADO

Ganho de latencia sem inconsistencias graves.

MODELO DE USO

/CACHE

Defina chave, TTL, estrategia de invalidacao, dados proibidos e comportamento em miss/stale.

EXEMPLO COMPLETO

/CACHE

Catalogo quase estatico: cache 10 min por tenant; invalidar apos atualizacao; nao cachear resposta personalizada sem chave de usuario/escopo.

Atencao Cache sem invalidacao e autorizacao correta pode vazar ou servir dado obsoleto.

Combine com /SECURITY-REVIEW + /OBSERVABILIDADE

CAPITULO 07

Seguranca, performance e observabilidade

Seguranca e confiabilidade devem aparecer no prompt desde o desenho, nao como checklist final. O GPT deve considerar fronteiras de confianca, autorizacao, segredos, logs, abuso, falhas parciais e evidencias para diagnostico.

/SECURITY-REVIEW

O QUE FAZ

Revisa superficies de ataque e controles necessarios.

QUANDO USAR

Antes de release, integracoes, autenticacao e dados sensiveis.

ENTRADAS MINIMAS

Fluxos, papeis, dados e fronteiras.

RESULTADO ESPERADO

Lista priorizada de riscos e mitigacoes.

MODELO DE USO

/SECURITY-REVIEW

Mapeie entrada, autenticacao, autorizacao, dados sensiveis, segredos, uploads, logs e abuso. Priorize riscos por impacto e probabilidade.

EXEMPLO COMPLETO

/SECURITY-REVIEW

Usuario comum nao pode alterar case de outra unidade mesmo mudando ID na URL. Verifique autorizacao no servidor em cada operacao, nao apenas visibilidade do botao.

Atencao Ocultar botao no frontend nao e autorizacao.

Combine com /OWASP + /TESTES

/OWASP

O QUE FAZ

Usa categorias conhecidas para ampliar a cobertura de revisao.

QUANDO USAR

Aplicacoes web e APIs.

ENTRADAS MINIMAS

Arquitetura, entradas e controles atuais.

RESULTADO ESPERADO

Cobertura estruturada sem checklist abstrato.

MODELO DE USO

/OWASP

Use OWASP como lente de cobertura, mas vincule cada risco ao fluxo real. Verifique controle de acesso, injecao, configuracao, auth, integridade e logging.

EXEMPLO COMPLETO

/OWASP

Para endpoint de consulta por ID, teste IDOR/BOLA; para filtros SQL, parametrizacao; para upload, tipo real, tamanho, armazenamento e execucao.

Atencao Nao rotule vulnerabilidade sem evidencia; diferencie hipotese de achado confirmado.

Combine com /SECURITY-REVIEW

/OBSERVABILIDADE

O QUE FAZ

Planeja logs, metricas e traces uteis.

QUANDO USAR

APIs, jobs, falhas intermitentes e operacao.

ENTRADAS MINIMAS

Sinais, IDs de correlacao e SLOs.

RESULTADO ESPERADO

Diagnostico mais rapido e visibilidade operacional.

MODELO DE USO

/OBSERVABILIDADE

Defina eventos de log estruturado, metricas de volume/erro/latencia e tracing entre dependencias. Remova dados sensiveis.

EXEMPLO COMPLETO

/OBSERVABILIDADE

Cada request recebe traceId; loga rota, resultado, latencia e dependency status; metrica p95; nenhum token ou senha em log.

Atencao Logar tudo sem estrutura gera ruido e risco de exposicao.

Combine com /LOGGING + /PERFORMANCE-WEB

/LOGGING

O QUE FAZ

Define eventos e campos com valor diagnostico.

QUANDO USAR

Backend, jobs e agentes.

ENTRADAS MINIMAS

Eventos, severidade e dados permitidos.

RESULTADO ESPERADO

Logs pesquisaveis e seguros.

MODELO DE USO

/LOGGING

Use logs estruturados com timestamp, level, event, traceId, entityId seguro e contexto minimo. Redija dados sensiveis.

EXEMPLO COMPLETO

/LOGGING

```
{level:"WARN",event:"api_dependency_timeout",traceId:"...",dependency:"civil_lookup",elapsedMs:3012}
```

Atencao Nao coloque CPF completo, senha, cookie ou token de acesso em log.

Combine com /OBSERVABILIDADE

/RESILIENCIA

O QUE FAZ

Desenha comportamento sob falha de dependencias.

QUANDO USAR

Sistemas distribuidos e APIs externas.

ENTRADAS MINIMAS

Tipo de falha, criticidade e alternativas.

RESULTADO ESPERADO

Sistema degrada de forma controlada.

MODELO DE USO

/RESILIENCIA

Classifique falhas transitórias vs permanentes. Defina timeout, retry seletivo, circuit breaker, fallback e idempotencia.

EXEMPLO COMPLETO

/RESILIENCIA

Se servico de enriquecimento falhar, cadastro continua com status "pendente de consulta"; retry em fila; operacao principal nao duplica.

Atencao Fallback silencioso pode mascarar dados incorretos; sinalize estado degradado.

Combine com /INTEGRACAO-API + /OBSERVABILIDADE

CAPITULO 08

Testes, debugging, code review e refatoracao

A qualidade do resultado melhora quando o prompt pede evidencia. Testes devem cobrir comportamento, regressao e limites; debugging deve separar sintoma, hipotese, experimento e causa.

/TESTES

O QUE FAZ

Define estrategia de verificacao por risco.

QUANDO USAR

Toda implementacao e correcao.

ENTRADAS MINIMAS

Criterios de aceite, fluxos e riscos.

RESULTADO ESPERADO

Suite alinhada ao comportamento esperado.

MODELO DE USO

/TESTES

Derive testes dos criterios de aceite. Cubra happy path, limites, erro, autorizacao e regressao.

EXEMPLO COMPLETO

/TESTES

Cadastro: salvar valido; obrigatorio ausente; formato invalido; duplicidade; usuario sem permissao; timeout externo; reabrir e confirmar persistencia.

Atencao Quantidade de testes nao substitui cobertura de risco.

Combine com /UNIT-TEST + /E2E

/UNIT-TEST

O QUE FAZ

Testa regra isolada e invariantes.

QUANDO USAR

Funcoes, servicos e validadores.

ENTRADAS MINIMAS

Contrato da unidade e dependencias.

RESULTADO ESPERADO

Feedback rapido sobre logica.

MODELO DE USO

/UNIT-TEST

Teste entradas representativas, bordas e erros; isole somente dependencias externas necessarias.

EXEMPLO COMPLETO

/UNIT-TEST

ApproveCase: permitido em OPEN; rejeita CLOSED; rejeita usuario sem role; grava evento uma vez.

Atencao Mockar tudo pode testar implementacao em vez de comportamento.

Combine com /BACKEND

/E2E

O QUE FAZ

Valida fluxo de ponta a ponta em ambiente realista.

QUANDO USAR

Fluxos criticos e regressao visual/funcional.

ENTRADAS MINIMAS

Persona, pre-condicao, passos e resultado.

RESULTADO ESPERADO

Confianca no caminho critico.

MODELO DE USO

/E2E

Defina cenarios por jornada, seletores estaveis, dados controlados e asserts no resultado observavel.

EXEMPLO COMPLETO

/E2E

Login -> abrir caso -> editar participante -> salvar -> recarregar -> confirmar valor e evento de historico.

Atencao Nao use E2E para toda combinacao; deixe regras pequenas nos testes unitarios.

Combine com /VISUAL-QA

/DEBUG

O QUE FAZ

Estrutura investigacao de falhas.

QUANDO USAR

Erros, regressao, 4xx/5xx, comportamento intermitente.

ENTRADAS MINIMAS

Sintoma, log, reproducao, mudancas recentes.

RESULTADO ESPERADO

Causa raiz sustentada por evidencia.

MODELO DE USO

```
/DEBUG
1 reproduza
2 delimite camada
3 formule hipoteses ordenadas
4 teste uma variavel
5 confirme causa
6 corrija
7 crie teste de regressao
```

EXEMPLO COMPLETO

```
/DEBUG
Sintoma: POST retorna 403, GET da tela funciona. Verifique WAF/proxy/CSRF/auth/rota;
compare request que funciona; nao altere credencial antes de localizar a camada que nega.
```

Atencao Nao confunda correlacao com causa; uma mudanca recente e pista, nao prova.

Combine com /OBSERVABILIDADE + /ANTI-ALUCINACAO

/CODE-REVIEW

O QUE FAZ

Revisa diff por corretude, risco e manutencao.

QUANDO USAR

Pull requests e patches gerados por IA.

ENTRADAS MINIMAS

Diff, requisitos e testes.

RESULTADO ESPERADO

Feedback acionavel e priorizado.

MODELO DE USO

`/CODE-REVIEW`
Priorize bugs, seguranca, regressao e contrato. Depois legibilidade. Cite arquivo/trecho e impacto.

EXEMPLO COMPLETO

`/CODE-REVIEW`
Verifique se nova validacao quebra edicao de registros antigos, se endpoint manteve autorizacao e se teste cobre caso de erro.

Atencao Review que so comenta estilo perde o que mais importa.

Combine com `/SECURITY-REVIEW + /TESTES`

/REFACTOR

O QUE FAZ

Melhora estrutura preservando comportamento externo.

QUANDO USAR

Código duplicado, acoplado ou difícil de testar.

ENTRADAS MINIMAS

Comportamento atual e testes de caracterização.

RESULTADO ESPERADO

Código mais simples com risco controlado.

MODELO DE USO

/REFACTOR

Documente comportamento antes. Refatore em passos pequenos; mantenha contrato; rode testes a cada passo.

EXEMPLO COMPLETO

/REFACTOR

Extraia validação duplicada para função pura, depois centralize acesso a API; não reescreva tela inteira no mesmo commit.

Atenção Refatoração misturada com mudança funcional dificulta diagnóstico.

Combine com /CLEAN-CODE + /TESTES

/CLEAN-CODE

O QUE FAZ

Aplica clareza, coesao e nomes orientados ao dominio.

QUANDO USAR

Apos corretude e arquitetura estarem definidas.

ENTRADAS MINIMAS

Convencoes do projeto e responsabilidade de cada unidade.

RESULTADO ESPERADO

Codigo mais legivel e modificavel.

MODELO DE USO

/CLEAN-CODE

Prefira nomes do dominio, funcoes curtas e coesas, fluxo explicito e comentarios que expliquem por que, nao o obvio.

EXEMPLO COMPLETO

/CLEAN-CODE

Troque processData por calculateRiskScore se essa e a responsabilidade real; remova flag booleana ambigua criando funcoes distintas.

Atencao "Clean" nao deve virar reescrita estetica de codigo estavel sem beneficio.

Combine com /CODE-REVIEW

/DOCUMENTAR

O QUE FAZ

Cria documentacao orientada a manutencao.

QUANDO USAR

Features, APIs, scripts e decisoes nao obvias.

ENTRADAS MINIMAS

Publico, nivel e fonte da verdade.

RESULTADO ESPERADO

Documentacao utilizavel por outro desenvolvedor.

MODELO DE USO

/DOCUMENTAR

Explique objetivo, pre-requisitos, fluxo, configuracao, exemplos, erros comuns e como validar. Nao documente comportamento inexistente.

EXEMPLO COMPLETO

/DOCUMENTAR

README do modulo: por que existe, env vars sem segredos, exemplo de chamada, codigos de erro, troubleshooting e comando de teste.

Atencao Copiar codigo para a documentacao sem contexto nao cria conhecimento.

Combine com /FONTE-DA-VERDADE

CAPITULO 09

Arquitetura, Git, CI/CD e operacao

Prompts de arquitetura precisam explicitar restricoes e trade-offs. O GPT deve apresentar alternativas, impactos e reversibilidade; nao escolher tecnologia apenas por popularidade.

/ARQUITETURA

O QUE FAZ

Define fronteiras, dependencias e fluxo de dados.

QUANDO USAR

Novos modulos, modernizacao ou problemas de acoplamento.

ENTRADAS MINIMAS

Requisitos, restricoes, escala e operacao.

RESULTADO ESPERADO

Desenho proporcional ao problema.

MODELO DE USO

/ARQUITETURA

Proponha componentes, responsabilidades, contratos e fluxo. Compare alternativas e justifique a menor arquitetura que atende os requisitos.

EXEMPLO COMPLETO

/ARQUITETURA

Sistema interno com 500 usuarios, banco Oracle existente e APEX: priorize modularidade e integracao simples; nao introduza microservicos sem necessidade.

Atencao Arquitetura sofisticada sem necessidade aumenta custo operacional.

Combine com /TRADEOFFS + /ADR

/TRADEOFFS

O QUE FAZ

Compara opcoes de forma explicita.

QUANDO USAR

Decisoes de framework, cache, banco, arquitetura ou UX.

ENTRADAS MINIMAS

Opcoes, criterios e pesos.

RESULTADO ESPERADO

Decisao transparente e defensavel.

MODELO DE USO

/TRADEOFFS

Compare A vs B por complexidade, risco, custo, performance, equipe, reversibilidade e compatibilidade.

EXEMPLO COMPLETO

/TRADEOFFS

CSS nativo vs nova biblioteca: CSS nativo ganha em compatibilidade e custo; biblioteca so vence se o projeto ja a usa ou padronizacao justificar.

Atencao Tabela de pros/contras sem contexto pode criar falsa equivalencia.

Combine com /ADR

/ADR

O QUE FAZ

Registra uma decisao arquitetural e suas consequencias.

QUANDO USAR

Decisoes de longo prazo ou dificil reversao.

ENTRADAS MINIMAS

Contexto, decisao, alternativas e consequencias.

RESULTADO ESPERADO

Historico de decisao compreensivel.

MODELO DE USO

```
/ADR
Titulo
Status
Contexto
Decisao
Alternativas
Consequencias
Plano de revisao
```

EXEMPLO COMPLETO

```
/ADR
Decisao: manter Oracle APEX Universal Theme e CSS local, evitando framework externo.
Motivo: reduzir conflito, carga e manutencao.
```

Atencao ADR nao deve justificar decisao depois do fato sem registrar os trade-offs reais.

Combine com /TRADEOFFS

/GIT-WORKFLOW

O QUE FAZ

Organiza commits, branches e revisao.

QUANDO USAR

Times e agentes que alteram repositorio.

ENTRADAS MINIMAS

Politica atual e tipo de mudanca.

RESULTADO ESPERADO

Historico facil de revisar e reverter.

MODELO DE USO

/GIT-WORKFLOW

Crie branch curta, commits atomicos, mensagem orientada ao motivo e PR com risco/testes. Nao misture refactor e feature sem necessidade.

EXEMPLO COMPLETO

/GIT-WORKFLOW

fix/form-actions-alignment; commit 1 CSS isolado; commit 2 teste visual; PR descreve antes/depois e rollback.

Atencao Commit gigante reduz rastreabilidade.

Combine com /ROLLBACK + /CODE-REVIEW

/CI-CD

O QUE FAZ

Automatiza verificacao e entrega.

QUANDO USAR

Projetos com releases frequentes ou risco de regressao.

ENTRADAS MINIMAS

Build, testes, ambientes e gates.

RESULTADO ESPERADO

Entrega repetivel e auditavel.

MODELO DE USO

/CI-CD

Defina pipeline: lint -> unit -> integration -> build -> security checks -> deploy stage
-> smoke -> prod com gate apropriado.

EXEMPLO COMPLETO

/CI-CD

PR roda lint/unit; merge gera artefato imutavel; stage recebe o mesmo artefato; smoke
passa antes de producao; rollback reutiliza versao anterior.

Atencao Recompilar diferente por ambiente pode gerar divergencia.

Combine com /TESTES + /ROLLBACK

/ROLLBACK

O QUE FAZ

Exige estratégia de reversão antes da mudança.

QUANDO USAR

Alteração de schema, deploy, agente autônomo e produção.

ENTRADAS MINIMAS

Artefato anterior, banco e dados afetados.

RESULTADO ESPERADO

Risco operacional reduzido.

MODELO DE USO

/ROLLBACK

Antes de executar, descreva como reverter código, configuração e dados. Pare se a mudança não for reversível sem aprovação.

EXEMPLO COMPLETO

/ROLLBACK

CSS: remover arquivo/commit. Banco: migration down apenas se segura; caso contrário usar expand-contract e feature flag.

Atenção Nem toda migração de dados é reversível; nesse caso, a estratégia deve ser roll-forward controlada.

Combine com /CI-CD + /MIGRAÇÃO

CAPITULO 10

Oracle APEX - desenvolvimento assistido por GPT

No APEX, um bom prompt precisa distinguir Universal Theme, Page Designer, Regions, Items, Buttons, Dynamic Actions, Processes, Branches, Validations, Session State, SQL e JavaScript. O maior risco e modernizar a aparencia quebrando metadados ou comportamento declarativo.

/APEX-UIUX

O QUE FAZ

Aplica UI/UX respeitando o modelo declarativo do APEX.

QUANDO USAR

Modernizacao de pagina APEX existente.

ENTRADAS MINIMAS

App/page, tipo de regioao, itens e objetivo.

RESULTADO ESPERADO

Pagina mais moderna com baixo risco de regressao.

MODELO DE USO

/APEX-UIUX /PRESERVAR-FUNCIONALIDADE
Analise Page Designer, Universal Theme e DOM gerado. Priorize propriedades nativas e classes antes de overrides invasivos.

EXEMPLO COMPLETO

/APEX-UIUX
Pagina 50, Form. Reagrupe campos em secoes, centralize acoes e melhore responsividade sem renomear itens, processes ou DAs.

Atencao Nao trate HTML gerado pelo APEX como HTML estatico que pode ser reescrito livremente.

Combine com /APEX-UNIVERSAL-THEME + /APEX-SEM-QUEBRAR

/APEX-FORMS

O QUE FAZ

Otimiza formulários APEX e seus estados.

QUANDO USAR

Form regions, CRUD e cadastro.

ENTRADAS MINIMAS

Region source, Items, PK, processos e validações.

RESULTADO ESPERADO

Form mais claro preservando persistência.

MODELO DE USO

/APEX-FORMS

Mapeie PK, source, items, validations, computations, DAs, processes e branches. Depois proponha layout e UX.

EXEMPLO COMPLETO

/APEX-FORMS

Formulário deve manter Automatic Row Processing. Agrupe itens sem alterar Source/Session State; testar create, edit e cancel.

Atenção Mover ou copiar Item sem entender source/session state pode causar valor nulo ou comportamento diferente.

Combine com /FORM-UX + /APEX-SESSION

/APEX-INTERACTIVE-REPORT

O QUE FAZ

Aprimora IR/IG com filtros e navegacao.

QUANDO USAR

Listagens, consulta e drill-down.

ENTRADAS MINIMAS

SQL source, colunas, links, filtros e destino.

RESULTADO ESPERADO

Navegacao de listagem para detalhe previsivel.

MODELO DE USO

/APEX-INTERACTIVE-REPORT

Revise SQL, colunas, link de linha, filtros, autorizacao e responsividade. Preserve aliases usados por links/formatacao.

EXEMPLO COMPLETO

/APEX-INTERACTIVE-REPORT

Na coluna ID, crie link para Page 50 passando P50_ID=&ID. com Clear Cache apropriado; confirmar session state e autorizacao.

Atencao Nao invente Page Item destino; confirme o nome real.

Combine com /APEX-SESSION + /ANTI-ALUCINACAO

/APEX-UNIVERSAL-THEME

O QUE FAZ

Prioriza templates, Theme Roller e classes suportadas.

QUANDO USAR

Alteracoes visuais em APEX.

ENTRADAS MINIMAS

Theme version, template e classes existentes.

RESULTADO ESPERADO

Menos CSS fragil e melhor compatibilidade com upgrades.

MODELO DE USO

/APEX-UNIVERSAL-THEME

Primeiro procure opcao de template, Template Options, grid e utilitarios do Universal Theme. So depois adicione CSS customizado.

EXEMPLO COMPLETO

/APEX-UNIVERSAL-THEME

Use t-Form-fieldContainer, grid e Template Options para spacing; CSS custom apenas para lacuna especifica.

Atencao Override profundo de seletores internos pode quebrar em atualizacao do tema.

Combine com /CSS-ARQUITETURA

/APEX-CSS

O QUE FAZ

Aplica CSS escopado ao APEX.

QUANDO USAR

Ajuste visual que nao existe nas opcoes nativas.

ENTRADAS MINIMAS

Static ID/region e seletor confirmado.

RESULTADO ESPERADO

Ajuste local e reversivel.

MODELO DE USO

/APEX-CSS

Escopo pelo Static ID da pagina/regiao. Evite seletores globais e classes internas instaveis quando houver alternativa.

EXEMPLO COMPLETO

/APEX-CSS

#case-form .t-ButtonRegion-wrap { justify-content:center; } com teste em desktop/mobile e foco.

Atencao Usar !important em cadeia e selecionar estrutura interna por nth-child e fragil.

Combine com /PRESERVAR-FUNCIONALIDADE + /VISUAL-QA

/APEX-JS

O QUE FAZ

Implementa comportamento cliente usando APIs e eventos apropriados do APEX.

QUANDO USAR

Dynamic Actions avancadas ou integracoes.

ENTRADAS MINIMAS

Items, eventos, APIs APEX e necessidade real.

RESULTADO ESPERADO

Codigo integrado ao runtime do APEX.

MODELO DE USO

/APEX-JS

Prefira Dynamic Action para regras simples; use apex.item/apex.server e eventos APEX quando JS for necessario.

EXEMPLO COMPLETO

/APEX-JS

Ao mudar P50_UF, Dynamic Action chama processo AJAX; JS atualiza P50_MUNICIPIO e trata erro. Nao use document.querySelector pelo id gerado se apex.item resolve.

Atencao JS direto no DOM pode ignorar refresh de regioao e session state.

Combine com /APEX-SESSION

/APEX-SESSION

O QUE FAZ

Controla passagem de valores, clear cache e Session State.

QUANDO USAR

Links entre paginas, filtros e processos.

ENTRADAS MINIMAS

Pagina origem/destino, itens e comportamento desejado.

RESULTADO ESPERADO

Navegacao sem valores stale ou filtros fantasmas.

MODELO DE USO

/APEX-SESSION

Mapeie valores enviados, itens destino, clear cache e quando o session state deve ser limpo ou preservado.

EXEMPLO COMPLETO

/APEX-SESSION

Page 28 -> Page 50: passar P50_ID com valor da linha; Clear Cache 50 somente quando abrir novo registro, nao ao editar sem necessidade.

Atencao Clear Cache incorreto pode apagar valores que o usuario esperava manter ou reutilizar session state antigo.

Combine com /APEX-INTERACTIVE-REPORT

/APEX-VALIDATION

O QUE FAZ

Revisa validacoes declarativas e mensagens.

QUANDO USAR

Forms e processos DML.

ENTRADAS MINIMAS

Itens, regra, timing e mensagem.

RESULTADO ESPERADO

Erros claros e fluxo preservado.

MODELO DE USO

/APEX-VALIDATION

Use validacao declarativa quando possivel; associe ao item; mensagem explica correcao; regra de integridade tambem existe no banco/backend quando necessario.

EXEMPLO COMPLETO

/APEX-VALIDATION

P50_DATA_FATO nao pode estar no futuro. Validacao PL/SQL/Expression apropriada com erro inline no item.

Atencao Validar so com JavaScript nao protege submit manipulado.

Combine com /FORM-UX + /SECURITY-REVIEW

/APEX-SECURITY

O QUE FAZ

Revisa authorization schemes, session e exposicao de dados.

QUANDO USAR

Paginas/regions/processes com perfis diferentes.

ENTRADAS MINIMAS

Papeis, pages, processes e queries.

RESULTADO ESPERADO

Controles aplicados no servidor e na UI.

MODELO DE USO

/APEX-SECURITY

Verifique Authentication, Authorization Scheme em pagina/regiao/botao/processo, protecao de session state e filtro de dados no servidor.

EXEMPLO COMPLETO

/APEX-SECURITY

Botao Excluir e processo Delete usam mesmo Authorization Scheme; SQL restringe unidade do usuario; nao confia em item hidden como controle de acesso.

Atencao Server-side authorization e obrigatoria; esconder componente e apenas UX.

Combine com /OWASP

/APEX-SEM-QUEBRAR

O QUE FAZ

Cria guardrails especificos para alteracao em Page Designer.

QUANDO USAR

Agente modificando pagina existente.

ENTRADAS MINIMAS

Itens/regions/processes que devem ser preservados.

RESULTADO ESPERADO

Mudancas menores, auditaveis e reversiveis.

MODELO DE USO

/APEX-SEM-QUEBRAR

Nao renomeie, delete ou recrie componentes sem necessidade. Registre antes/depois de propriedades alteradas e teste create/edit/navigation.

EXEMPLO COMPLETO

/APEX-SEM-QUEBRAR

Preserve P50_ID, Form Region, Automatic Row Processing, validations e branches. Ajustar somente grid/template/classes no primeiro ciclo.

Atencao Recriar regioao "para ficar mais limpa" pode perder configuracoes invisiveis.

Combine com /ROLLBACK + /APEX-AGENTE

/APEX-AGENTE

O QUE FAZ

Instrui um agente a operar Page Designer com checkpoints.

QUANDO USAR

Quando um agente tem acesso ao navegador/IDE.

ENTRADAS MINIMAS

URL/sessao, pagina, tarefa, permissao e criterios.

RESULTADO ESPERADO

Execucao controlada e rastreavel.

MODELO DE USO

```
/APEX-AGENTE
1 abra pagina alvo
2 inspecione antes
3 registre componentes
4 altere menor conjunto
5 execute pagina
6 teste criterios
7 se falhar, reverta
8 reporte exatamente o que mudou
```

EXEMPLO COMPLETO

```
/APEX-AGENTE
Abra App 107 Page 50 na sessao ja autenticada. Nao crie nova sessao. Inspecone Form
Region e Button Region, centralize botoes via propriedades/CSS escopado, execute, teste
com e sem registro e documente propriedades alteradas.
```

Atencao Nao autorize "melhorar o que achar necessario" em ambiente real sem limites.

Combine com /ANALISAR-ANTES + /VALIDAR-RESULTADO

CAPITULO 11

Agentes de desenvolvimento e execucao segura

Quando a IA deixa de apenas sugerir e passa a clicar, editar arquivos ou executar comandos, o prompt precisa funcionar como procedimento operacional. A autonomia deve vir acompanhada de escopo, checkpoints, verificacao e limites de parada.

/AGENTE-EXECUTOR

O QUE FAZ

Define comportamento de um agente que pode agir no ambiente.

QUANDO USAR

IDE, navegador, terminal ou Page Designer.

ENTRADAS MINIMAS

Objetivo, ambiente, permissao e limites.

RESULTADO ESPERADO

Agente com autonomia controlada.

MODELO DE USO

`/AGENTE-EXECUTOR`
Execute a tarefa no ambiente existente. Antes de qualquer escrita, leia e registre o estado. Nao ultrapasse o escopo. Pare em caso de ambiguidade destrutiva.

EXEMPLO COMPLETO

`/AGENTE-EXECUTOR`
Pode editar somente arquivos de frontend desta feature. Nao alterar banco, secrets ou pipeline. Rode testes afetados e apresente diff final.

Atencao Permissao total sem guardrails aumenta risco de dano silencioso.

Combine com `/ROLLBACK + /VALIDAR-RESULTADO`

/VERIFICAR-ANTES-DE-ALTERAR

O QUE FAZ

Exige prova do estado atual.

QUANDO USAR

Qualquer tarefa de modificacao por agente.

ENTRADAS MINIMAS

Artefatos e propriedades a confirmar.

RESULTADO ESPERADO

Reduz alteracao no alvo errado.

MODELO DE USO

/VERIFICAR-ANTES-DE-ALTERAR

Antes de editar, confirme arquivo/pagina correta, branch/sessao, dependencias, estado atual e backup/diff disponivel.

EXEMPLO COMPLETO

/VERIFICAR-ANTES-DE-ALTERAR

Confirme que Page Designer mostra App 107 Page 50 e que os botoes alvo sao SAVE/CANCEL antes de mudar alinhamento.

Atencao Nome parecido nao e confirmacao suficiente.

Combine com /AGENTE-EXECUTOR

/PASSO-A-PASSO

O QUE FAZ

Transforma tarefa complexa em checkpoints observaveis.

QUANDO USAR

Agentes e debugging.

ENTRADAS MINIMAS

Fases e criterios de conclusao por fase.

RESULTADO ESPERADO

Execucao mais auditavel.

MODELO DE USO

/PASSO-A-PASSO

Para cada etapa: objetivo -> acao -> verificacao -> resultado. Nao avance se a verificacao falhar.

EXEMPLO COMPLETO

/PASSO-A-PASSO

Etapa 1 localizar regioao; verificar Static ID. Etapa 2 aplicar classe; verificar DOM. Etapa 3 executar; verificar foco, alinhamento e submit.

Atencao Passos demais podem virar burocracia; use checkpoints nos pontos de risco.

Combine com /PLANO-EXECUCAO

/VALIDAR-RESULTADO

O QUE FAZ

Obriga a confrontar a saída com os criterios de aceite.

QUANDO USAR

Final de qualquer tarefa feita por IA/agente.

ENTRADAS MINIMAS

Criterios de aceite e evidencias.

RESULTADO ESPERADO

Relatorio de conclusao confiavel.

MODELO DE USO

/VALIDAR-RESULTADO

Para cada criterio: PASS/FAIL + evidencia. Se FAIL, nao declare concluido.

EXEMPLO COMPLETO

/VALIDAR-RESULTADO

PASS IDs preservados (diff). PASS 360px sem overflow (screenshot). PASS salvar/editar (teste). FAIL teclado no modal -> corrigir antes de concluir.

Atencao "Parece correto" nao e evidencia.

Combine com /CRITERIOS-ACEITE + /TESTES

/PARAR-SE

O QUE FAZ

Define condicoes de interrupcao para evitar decisoes destrutivas.

QUANDO USAR

Agentes com permissao de escrita.

ENTRADAS MINIMAS

Eventos que exigem aprovacao humana.

RESULTADO ESPERADO

Limite claro de autonomia.

MODELO DE USO

/PARAR-SE

Interrompa antes de excluir dados, alterar schema destrutivamente, expor segredo, trocar auth, modificar producao fora do escopo ou se a fonte da verdade divergir.

EXEMPLO COMPLETO

/PARAR-SE

Se para centralizar botoes for necessario recriar a Form Region ou mudar processo DML, pare e reporte a dependencia em vez de executar.

Atencao Sem condicao de parada, o agente tende a "resolver" expandindo escopo.

Combine com /ANTI-ALUCINACAO + /ROLLBACK

CAPITULO 12

Roteiros completos e exemplos complexos

Os exemplos a seguir demonstram como os comandos funcionam em conjunto. Cada roteiro tem objetivo, sequencia de trabalho, prompt mestre e criterios de aceite. Eles podem ser copiados e adaptados para uma IA conversacional ou agente com acesso ao projeto.

Roteiro 1 - Redesign completo de formulario legado

Comandos /ATUAR-COMO /CONTEXTO /FONTE-DA-VERDADE /UX-AUDIT /FORM-UX /UI-MODERNA /RESPONSIVO /ACESSIBILIDADE /PRESERVAR-FUNCIONALIDADE /TESTES /VALIDAR-RESULTADO

Sequencia recomendada

1. Inspecionar markup/configuracao e mapear campos, regras, eventos e dependencias.
2. Classificar problemas de UX por severidade e impacto.
3. Propor nova hierarquia e agrupamento sem alterar contrato de dados.
4. Implementar a menor mudanca visual possivel, preferindo recursos nativos do stack.
5. Testar validacoes, teclado, responsividade, salvar, editar, cancelar e estados de erro.
6. Comparar com criterios de aceite e registrar alteracoes.

PROMPT MESTRE - COPIAR E ADAPTAR

/ATUAR-COMO: UX Engineer + Front-End senior
/ANALISAR-ANTES /UX-AUDIT /FORM-UX /RESPONSIVO /ACESSIBILIDADE
/PRESERVAR-FUNCIONALIDADE /ANTI-ALUCINACAO /TESTES /VALIDAR-RESULTADO

CONTEXTO

Tenho um formulario legado de cadastro com aproximadamente 30 campos. Ele funciona e persiste corretamente, mas possui baixa hierarquia visual, botoes deslocados, espacamento irregular e experiencia ruim em telas menores.

FONTE DA VERDADE

Use exclusivamente os arquivos e configuracoes fornecidos. Nao invente IDs, eventos, endpoints ou campos.

OBJETIVO

Redesenhar a experiencia sem mudar contrato de dados ou regra de negocio. Agrupe campos por contexto, destaque a acao principal e preserve o fluxo atual.

RESTRICOES

- manter todos os IDs e nomes existentes;
- manter validacoes, listeners e submit;
- nao adicionar framework externo;
- CSS deve ser escapado ao componente/pagina.

CRITERIOS DE ACEITE

- sem overflow em 360px;

- navegacao completa por teclado;
- foco visivel;
- salvar/editar/cancelar inalterados;
- zero erros novos no console;
- nenhuma regra removida.

EXECUCAO

Primeiro entregue diagnostico e mapa de dependencias. Depois implemente. Ao final, informe PASS/FAIL para cada criterio e liste os arquivos/propriedades alterados.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 2 - Dashboard administrativo responsivo

Comandos /WEBDESIGNER /DASHBOARD-UI /HIERARQUIA-VISUAL /DESIGN-SYSTEM /CARDS /TABLE-UX /RESPONSIVO /PERFORMANCE-WEB /ACESSIBILIDADE

Sequencia recomendada

1. Definir perguntas que o dashboard precisa responder.
2. Separar indicadores, tendencias, alertas e lista operacional.
3. Definir tokens e componentes antes de estilizar cada bloco.
4. Criar comportamento responsivo por prioridade, nao por simples reducao.
5. Testar volume real, estados vazios, erro e carregamento.

PROMPT MESTRE - COPIAR E ADAPTAR

/WEBDESIGNER /UIUX-PRO /DESIGN-SYSTEM /RESPONSIVO /ACESSIBILIDADE /PERFORMANCE-WEB

Crie um dashboard administrativo para supervisores que precisam identificar rapidamente: volume total, itens criticos, pendencias por unidade e casos recentes.

HIERARQUIA

1. Alertas criticos e KPIs;
2. tendencia e distribuicao;
3. tabela operacional com filtros;
4. metadados secundarios.

REGRAS DE UI

- cards apenas para KPIs e agrupamentos reais;
- tabela deve permanecer operavel com 500+ linhas paginadas;
- status usa cor + texto/icone;
- loading, vazio, erro e sem permissao devem ter estados proprios;
- em mobile, priorize alertas, 3 KPIs e lista compacta.

ENTREGAVEIS

1 wireframe textual; 2 estrutura de componentes; 3 HTML/CSS/JS ou stack informada; 4 checklist ally; 5 plano de teste responsivo.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 3 - CRUD com API e banco

Comandos /ARQUITETURA /API-DESIGN /BACKEND /MODELAGEM-DADOS /SQL /SECURITY-REVIEW /TESTES /OBSERVABILIDADE /CI-CD

Sequencia recomendada

1. Modelar dominio e invariantes.
2. Definir contrato de API antes de implementar.
3. Definir autorizacao e transacoes.
4. Criar schema/migrations reversiveis.
5. Implementar casos de uso e persistencia.
6. Testar integracao e erros.
7. Adicionar logs/metricas e pipeline.

PROMPT MESTRE - COPIAR E ADAPTAR

/ATUAR-COMO: Engenheiro de Software senior
/ARQUITETURA /API-DESIGN /MODELAGEM-DADOS /SECURITY-REVIEW /TESTES /OBSERVABILIDADE

Feature: cadastro de casos e participantes. Um caso possui varios participantes com papeis diferentes. Mudancas de status devem ser historizadas.

Requisitos:

- API REST;
- autorizacao por unidade;
- criacao idempotente quando o cliente reenviar a mesma requisicao;
- historico imutavel de status;
- erros estruturados com traceId;
- transacao para alteracoes relacionadas;
- migrations sem quebra de versao anterior.

Entregue:

- 1 modelo de dados e invariantes;
- 2 endpoints e exemplos de payload;
- 3 estrategia de autorizacao;
- 4 pseudocodigo/implementacao das camadas;
- 5 SQL/migrations;
- 6 testes unitarios + integracao;
- 7 logs/metricas;
- 8 riscos e trade-offs.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 4 - Debug de erro HTTP 403 em autenticao

Comandos /DEBUG /ANTI-ALUCINACAO /OBSERVABILIDADE /SECURITY-REVIEW /INTEGRACAO-API /TESTES

Sequencia recomendada

1. Reproduzir e capturar request/response.

2. Delimitar se o 403 vem de navegador, proxy/WAF, servidor web, ORDS, aplicacao ou autenticacao.
3. Comparar metodo, rota, headers e cookies com chamada que funciona.
4. Testar hipoteses uma a uma, da camada externa para interna.
5. Corrigir a camada que realmente bloqueia e criar smoke test.

PROMPT MESTRE - COPIAR E ADAPTAR

/DEBUG /ANTI-ALUCINACAO /SECURITY-REVIEW /VALIDAR-RESULTADO

SINTOMA

A pagina GET de login abre, mas o POST usado para autenticar retorna HTTP 403.

NAO ASSUMA A CAUSA.

Primeiro identifique qual camada gera o 403. Analise status, headers, corpo, logs, proxy/WAF, rota, metodo, cookies, CSRF e configuracao do servidor.

ROTEIRO OBRIGATORIO

- 1 reproduzir com DevTools e registrar request;
- 2 comparar GET e POST;
- 3 localizar evidencia no log da camada que negou;
- 4 formular no maximo 3 hipoteses ordenadas;
- 5 testar uma por vez;
- 6 aplicar a menor correcao;
- 7 confirmar login completo;
- 8 criar teste que detecte regressao do POST.

Se nao houver evidencia suficiente, diga exatamente qual log/configuracao falta. Nao invente regra de firewall, endpoint ou causa.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 5 - Auditoria de acessibilidade de uma aplicacao

Comandos /ACESSIBILIDADE /HTML-SEMANTICO /FORM-UX /BUTTON-UX /VISUAL-QA /TESTES

Sequencia recomendada

1. Mapear jornadas criticas.
2. Testar teclado e foco.
3. Revisar nomes acessiveis, labels e erros.
4. Revisar contraste, zoom/reflow e movimento.
5. Classificar achados por severidade e impacto.
6. Criar testes de regressao.

PROMPT MESTRE - COPIAR E ADAPTAR

/ACESSIBILIDADE /UX-AUDIT /VISUAL-QA /TESTES

Audite a interface fornecida considerando uso apenas por teclado, leitor de tela, zoom 200%, reflow em 320 CSS px, contraste e reduced motion.

Para cada achado informe:

- severidade: bloqueador/alto/medio/baixo;

- componente;
- comportamento atual;
- impacto;
- correcao recomendada;
- como testar manualmente;
- teste automatizavel quando aplicavel.

Priorize primeiro problemas que impedem concluir a tarefa. Nao limite a auditoria a atributos ARIA.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 6 - Performance de pagina lenta

Comandos /PERFORMANCE-WEB /OBSERVABILIDADE /SQL /CACHE /TESTES

Sequencia recomendada

1. Definir baseline e metrica alvo.
2. Separar tempo de servidor, rede, JS, renderizacao e imagens.
3. Encontrar top 3 gargalos por evidencia.
4. Aplicar uma mudanca por vez.
5. Repetir medicao e verificar regressao funcional.

PROMPT MESTRE - COPIAR E ADAPTAR

/PERFORMANCE-WEB /OBSERVABILIDADE /ANALISAR-ANTES

Problema: pagina leva 6-9s para ficar utilizavel.

Nao proponha otimizacoes genericas. Primeiro construa um diagnostico por camadas: 1 TTFB/backend; 2 consultas SQL; 3 rede; 4 bundle/JS; 5 renderizacao/DOM; 6 imagens; 7 chamadas duplicadas.

Para cada hipotese, diga a evidencia necessaria e como medir. Depois priorize por impacto x esforco.

Criterio de aceite: p75 de carregamento percebido abaixo de <meta>, sem alterar corretude dos dados e sem esconder loading de forma enganosa.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 7 - Code review de patch gerado por IA

Comandos /CODE-REVIEW /SECURITY-REVIEW /TESTES /PRESERVAR-FUNCIONALIDADE /ANTI-ALUCINACAO

Sequencia recomendada

1. Ler requisitos e diff.
2. Revisar comportamento alterado e contrato.
3. Buscar bugs, auth, dados, concorrência e regressão.
4. Verificar testes e casos ausentes.
5. Somente depois comentar estilo/manutenção.

PROMPT MESTRE - COPIAR E ADAPTAR

/CODE-REVIEW /SECURITY-REVIEW /PRESERVAR-FUNCIONALIDADE

Revise o diff fornecido como gate de produção. Não reescreva o código inteiro.

Classifique findings:

P0 bloqueador; P1 alto; P2 medio; P3 melhoria.

Para cada finding: arquivo/trecho, problema, cenário concreto de falha, correção mínima e teste que comprova.

Revise especialmente:

- controle de acesso;
- validação de entrada;
- alteração de contrato/API;
- regressão em dados antigos;
- concorrência/idempotência;
- tratamento de erro;
- cobertura de teste.

Se não houver finding relevante, declare isso sem inventar problema estético.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnóstico e premissas; depois permita implementação; por fim exija testes e evidências de aceite. Isso evita que uma resposta longa esconda uma decisão incorreta no início.

Roteiro 8 - Modernização de sistema legado sem reescrita

Comandos /LEGACY-MODERNIZE /ARQUITETURA /PRESERVAR-FUNCIONALIDADE /REFACTOR /MIGRACAO /TESTES /ROLLBACK

Sequencia recomendada

1. Inventariar módulos, dependências e riscos.
2. Criar testes de caracterização.
3. Escolher seams/fronteiras para extração gradual.
4. Modernizar por fatias verticais.
5. Manter compatibilidade e rollback por etapa.

PROMPT MESTRE - COPIAR E ADAPTAR

/ARQUITETURA /REFACTOR /PRESERVAR-FUNCIONALIDADE /ROLLBACK

Objetivo: modernizar uma aplicação legada em operação sem big-bang rewrite.

Primeiro produza um mapa de risco: módulos, dependências, contratos, banco, integrações, áreas sem teste e pontos com maior mudança.

Proponha estrategia incremental:

- testes de caracterizacao;
- encapsular integracoes;
- reduzir acoplamento;
- extrair componentes por valor de negocio;
- manter contrato antigo durante transicao;
- medir regressao;
- rollback por release.

Nao sugira microservicos ou troca de framework sem justificativa baseada nas restricoes e capacidade operacional da equipe.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 9 - Oracle APEX: formulario moderno com agente

Comandos /APEX-AGENTE /APEX-UIUX /APEX-FORMS /APEX-UNIVERSAL-THEME /APEX-CSS /APEX-SEM-QUEBRAR /VALIDAR-RESULTADO

Sequencia recomendada

1. Confirmar app/page/sessao.
2. Inventariar Regions, Items, Buttons, DAs, Processes e Validations.
3. Registrar estado antes.
4. Priorizar Template Options/grid.
5. Aplicar CSS escopado somente se necessario.
6. Executar e testar create/edit/cancel/mobile.
7. Registrar propriedades alteradas e evidencias.

PROMPT MESTRE - COPIAR E ADAPTAR

```
/APEX-AGENTE /ANALISAR-ANTES /APEX-UIUX /APEX-FORMS
/APEX-UNIVERSAL-THEME /APEX-SEM-QUEBRAR /RESPONSIVO /ACESSIBILIDADE
/VALIDAR-RESULTADO
```

Use a sessao APEX ja autenticada e trabalhe somente na pagina informada.

ANTES DE EDITAR

- confirme App ID e Page ID;
- liste Regions, Items, Buttons, Dynamic Actions, Processes, Validations e Branches afetados;
- identifique o Form Region e a Button Region;
- registre propriedades atuais que serao alteradas.

OBJETIVO

Transformar o formulario em layout moderno, agrupado e responsivo, com botoes centralizados e hierarquia clara.

GUARDRAILS

Nao renomeie Items, nao recree Form Region, nao altere Automatic Row Processing, SQL, Session State ou regras de negocio sem necessidade comprovada. Prefira Universal Theme e Template Options; CSS apenas escopado ao Static ID.

TESTES

1 abrir novo; 2 editar existente; 3 validacao de obrigatorio; 4 salvar; 5 cancelar; 6 360px; 7 teclado/foco.

Ao final, apresente alteracoes exatas e PASS/FAIL.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 10 - Oracle APEX: navegar de relatorio para form filtrado

Comandos /APEX-INTERACTIVE-REPORT /APEX-SESSION /ANTI-ALUCINACAO /TESTES

Sequencia recomendada

1. Confirmar coluna origem e Item destino.
2. Definir target da coluna/botao.
3. Configurar Set Items / With Values.
4. Decidir Clear Cache corretamente.
5. Testar dois registros distintos e back navigation.

PROMPT MESTRE - COPIAR E ADAPTAR

/APEX-INTERACTIVE-REPORT /APEX-SESSION /ANTI-ALUCINACAO

Objetivo: ao clicar na acao "Cad. Envolvidos" da pagina de consulta, abrir a Page 50 ja carregando o registro correspondente.

Antes de configurar, confirme:

- coluna/valor chave da pagina origem;
- Page Item real da chave na Page 50;
- se a Page 50 e Form baseado nessa PK;
- processos que carregam o registro;
- comportamento esperado de Clear Cache.

Configure o target usando os nomes confirmados. Nao invente P50_ID se o item tiver outro nome.

Teste com registro A e B em sequencia para garantir que Session State nao reutilize valor anterior. Informe exatamente Target, Items to Set, Values e Clear Cache usados.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 11 - Feature completa do ticket ao deploy

Comandos /OBJETIVO /CRITERIOS-ACEITE /ARQUITETURA /PLANO-EXECUCAO /GIT-WORKFLOW /TESTES /SECURITY-REVIEW /CI-CD /VALIDAR-RESULTADO

Sequencia recomendada

1. Traduzir ticket em comportamento e aceite.

2. Mapear impacto e arquitetura.
3. Planejar commits e migração.
4. Implementar por fatias testáveis.
5. Executar testes/security review.
6. Preparar PR e deploy.
7. Smoke test e monitoramento pos-deploy.

PROMPT MESTRE - COPIAR E ADAPTAR

```
/ATUAR-COMO: Tech Lead
/OBJETIVO /CRITERIOS-ACEITE /ARQUITETURA /PLANO-EXECUCAO
/GIT-WORKFLOW /TESTES /SECURITY-REVIEW /CI-CD /VALIDAR-RESULTADO
```

Receba o ticket e transforme em um plano de entrega de producao.

Saida obrigatoria:

- 1 requisitos funcionais e nao funcionais;
- 2 perguntas/lacunas criticas;
- 3 impacto em frontend/backend/banco;
- 4 desenho da solucao com trade-offs;
- 5 plano de implementacao por commits pequenos;
- 6 testes derivados de cada criterio de aceite;
- 7 riscos de seguranca e dados;
- 8 migracao/rollback;
- 9 checklist de PR;
- 10 plano de deploy, smoke test e metricas pos-release.

Nao escreva codigo antes de fechar as premissas que podem mudar a arquitetura. Se a tarefa estiver suficientemente definida, avance sem criar perguntas artificiais.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

Roteiro 12 - Criar um Design System para sistema corporativo

Comandos /DESIGN-SYSTEM /UI-MODERNA /ACESSIBILIDADE /COMPONENTIZAR
/DARK-MODE /DOCUMENTAR

Sequencia recomendada

1. Inventariar componentes existentes.
2. Definir tokens semanticos.
3. Definir primitives e componentes compostos.
4. Especificar estados e acessibilidade.
5. Criar exemplos e governance.
6. Migrar gradualmente.

PROMPT MESTRE - COPIAR E ADAPTAR

```
/DESIGN-SYSTEM /ACESSIBILIDADE /COMPONENTIZAR /DOCUMENTAR
```

Crie a especificacao de um Design System para uma aplicacao corporativa administrativa.

Inclua:

- tokens: color, typography, spacing, radius, elevation, motion;
- cores semanticas e temas claro/escuro;
- Button, Input, Select, Checkbox, Radio, Textarea, Card, Alert, Badge, Table, Modal, Tabs, Breadcrumb, Pagination;
- estados: default/hover/focus/active/disabled/loading/error/success;
- regras de densidade para telas administrativas;
- acessibilidade e teclado por componente;
- exemplos de uso correto/incorreto;
- estrategia de versionamento e deprecacao;
- plano incremental para migrar telas legadas.

Evite componentes genericos sem necessidade e defina APIs pequenas.

Uso profissional Execute o prompt em ciclos. Primeiro valide diagnostico e premissas; depois permita implementacao; por fim exija testes e evidencias de aceite. Isso evita que uma resposta longa esconda uma decisao incorreta no inicio.

CAPITULO 13

Biblioteca rapida de comandos e combinacoes

Esta secao funciona como folha de consulta. Use uma macro so quando ela acrescentar uma regra operacional real. Empilhar 20 comandos sem contexto nao melhora automaticamente o resultado.

COMANDO	USO PRINCIPAL
/ATUAR-COMO	Define papel tecnico.
/OBJETIVO	Define estado desejado.
/CONTEXTO	Fornece ambiente e dependencias.
/FONTE-DA-VERDADE	Limita evidencias permitidas.
/RESTRICOES	Define limites e proibicoes.
/CRITERIOS-ACEITE	Define Definition of Done.
/ANTI-ALUCINACAO	Proibe inventar fatos/configuracoes.
/ANALISAR-ANTES	Diagnostico antes da escrita.
/PLANO-EXECUCAO	Fases e checkpoints.
/PRESERVAR-FUNCIONALIDADE	Evita regressao funcional.
/WEBDESIGNER	Web design + front-end.
/UIUX-PRO	UI e UX integrados.

COMANDO	USO PRINCIPAL
/UX-AUDIT	Auditoria de usabilidade.
/UI-MODERNA	Linguagem visual contemporanea.
/DESIGN-PREMIUM	Refinamento visual sobrio.
/HIERARQUIA-VISUAL	Prioridade visual e informacional.
/DESIGN-SYSTEM	Tokens e componentes.

/CARDS	Agrupamento modular.
/FORM-UX	Formulario orientado a tarefa.
/TABLE-UX	Tabela operacional.
/BUTTON-UX	Hierarquia de acoes.
/FEEDBACK-UX	Loading, sucesso e erro.
/MICROINTERACOES	Transicoes funcionais.
/RESPONSIVO	Layout adaptativo.

COMANDO	USO PRINCIPAL
/MOBILE-FIRST	Prioridade em telas pequenas.
/ACESSIBILIDADE	Teclado, semantica, contraste e estados.
/DARK-MODE	Tema escuro por tokens.
/VISUAL-QA	Checklist de regressao visual.
/HTML-SEMANTICO	Markup por significado.
/CSS-ARQUITETURA	CSS escopado e manutenivel.
/JAVASCRIPT	Logica cliente robusta.
/TYPESCRIPT	Contratos e estados tipados.
/COMPONENTIZAR	Reuso com coesao.
/STATE-MANAGEMENT	Fonte de verdade de estado.
/VALIDACAO-FRONTEND	Feedback de entrada no cliente.
/PERFORMANCE-WEB	Diagnostico e otimizacao.

COMANDO	USO PRINCIPAL
/SEO-TECNICO	Indexacao e metadados.
/API-DESIGN	Contrato de API.
/INTEGRACAO-API	Consumo resiliente.
/BACKEND	Casos de uso e regras.

/MODELAGEM-DADOS	Entidades e invariantes.
/SQL	Consulta correta e eficiente.
/MIGRACAO	Evolucao segura de schema.
/CACHE	Cache e invalidacao.
/SECURITY-REVIEW	Ameacas e controles.
/OWASP	Cobertura de riscos web.
/OBSERVABILIDADE	Logs, metricas e traces.
/LOGGING	Eventos estruturados.

COMANDO	USO PRINCIPAL
/RESILIENCIA	Timeout, retry e fallback.
/TESTES	Estrategia por risco.
/UNIT-TEST	Regras isoladas.
/E2E	Jornada critica.
/DEBUG	Causa raiz por evidencias.
/CODE-REVIEW	Revisao por risco.
/REFACTOR	Melhoria estrutural segura.
/CLEAN-CODE	Clareza e coesao.
/DOCUMENTAR	Documentacao operacional.
/ARQUITETURA	Fronteiras e fluxo.
/TRADEOFFS	Comparacao contextual.
/ADR	Registro de decisao.

COMANDO	USO PRINCIPAL
/GIT-WORKFLOW	Commits e PRs.
/CI-CD	Pipeline de entrega.
/ROLLBACK	Reversao/roll-forward.

/APEX-UIUX	UI/UX em APEX.
/APEX-FORMS	Forms e DML.
/APEX-INTERACTIVE-REPORT	IR/IG e navegacao.
/APEX-UNIVERSAL-THEME	Recursos nativos do tema.
/APEX-CSS	CSS escopado em APEX.
/APEX-JS	APIs JS e Dynamic Actions.
/APEX-SESSION	Session State e passagem de itens.
/APEX-VALIDATION	Validacoes APEX.
/APEX-SECURITY	AuthN/AuthZ e session.

COMANDO	USO PRINCIPAL
/APEX-SEM-QUEBRAR	Guardrail de metadados.
/APEX-AGENTE	Agente operando Page Designer.
/AGENTE-EXECUTOR	Autonomia com limites.
/VERIFICAR-ANTES-DE-ALTERAR	Check do alvo antes de escrever.
/PASSO-A-PASSO	Checkpoints de execucao.
/VALIDAR-RESULTADO	PASS/FAIL por criterio.
/PARAR-SE	Condicoes de interrupcao.

Combinacoes prontas

Redesign seguro /WEBDESIGNER /UX-AUDIT /UIUX-PRO /RESPONSIVO
/ACESSIBILIDADE /PRESERVAR-FUNCIONALIDADE /TESTES

Formulario profissional /FORM-UX /HIERARQUIA-VISUAL /BUTTON-UX /FEEDBACK-
UX /VALIDACAO-FRONTEND /ACESSIBILIDADE

Dashboard /UIUX-PRO /DESIGN-SYSTEM /CARDS /TABLE-UX /RESPONSIVO
/PERFORMANCE-WEB

API de producao /API-DESIGN /SECURITY-REVIEW /RESILIENCIA

/OBSERVABILIDADE /TESTES

Refatoracao segura /ANALISAR-ANTES /REFACTOR /PRESERVAR-FUNCIONALIDADE /TESTES /CODE-REVIEW /ROLLBACK

Debug serio /DEBUG /ANTI-ALUCINACAO /FONTE-DA-VERDADE /OBSERVABILIDADE /TESTES

Agente autonomo controlado /AGENTE-EXECUTOR /VERIFICAR-ANTES-DE-ALTERAR /PASSO-A-PASSO /PARAR-SE /ROLLBACK /VALIDAR-RESULTADO

Oracle APEX Forms /APEX-AGENTE /APEX-FORMS /APEX-UIUX /APEX-UNIVERSAL-THEME /APEX-SEM-QUEBRAR /APEX-SESSION /TESTES

Oracle APEX IR -> Form /APEX-INTERACTIVE-REPORT /APEX-SESSION /ANTI-ALUCINACAO /VALIDAR-RESULTADO

Template universal de prompt para desenvolvimento

PROMPT UNIVERSAL

/ATUAR-COMO: <papel>
/ANALISAR-ANTES /ANTI-ALUCINACAO /PRESERVAR-FUNCIONALIDADE
<adicione macros especificas da tarefa>

OBJETIVO
<estado atual -> estado desejado>

CONTEXT0
- Aplicacao/stack: <...>
- Pagina/modulo/arquivo: <...>
- Usuario e fluxo: <...>
- Problema observado: <...>

FONTE DA VERDADE
Use apenas <arquivos, codigo, schema, logs, documentacao>.
Nao invente nomes, IDs, rotas, tabelas ou configuracoes.

RESTRICOES
- preservar <contratos, IDs, regras, dados>;
- nao usar <tecnologia/procedimento proibido>;
- menor mudanca segura primeiro.

ROTEIRO
1 diagnosticar;
2 mapear dependencias e riscos;
3 propor solucao e trade-offs;
4 implementar;
5 testar;
6 validar criterios;
7 documentar mudancas e rollback.

CRITERIOS DE ACEITE
- <critério verificavel 1>
- <critério verificavel 2>

- <critério verificável 3>

ENTREGAVEIS

<codigo/diff/configuracao/testes/documentacao>

FINALIZACAO

Apresente PASS/FAIL por critério e evidencias. Não declare concluído se algum critério crítico falhar.

Checklist antes de enviar qualquer prompt complexo

- O objetivo descreve um resultado observável?
- A IA recebeu o contexto técnico que muda a decisão?
- A fonte da verdade está definida?
- Está claro o que não pode ser alterado?
- Os nomes/IDs fornecidos são reais e confirmados?
- Há critérios de aceite verificáveis?
- O prompt pede análise antes de mudança quando existe risco?
- Há testes para happy path, erro e regressão?
- Existe rollback ou estratégia de reversão?
- Agentes possuem condições explícitas de parada?
- A resposta final precisa comprovar o resultado em vez de apenas afirmar que terminou?

Princípio final Use GPT como acelerador de engenharia, não como substituto de evidência. Bons comandos criam contexto, limites e verificação. A qualidade real vem do ciclo: especificar -> analisar -> implementar -> testar -> validar -> registrar.

Fim do guia